CrossTwin: Function and Equation Registry

A traceability record between the DPSIR causal graph and its computational implementation

CrossTwin Project

November 12, 2026

Abstract

This document catalogs every quantitative computation currently implemented in the CrossTwin platform's domain apps (`administrative`, `physicalEnv`, `watersupply`, `housing`, `urban_heat`), plus the map-editing and what-if backup functions of `mainMap`, records the exact equation each function evaluates, the variables and their units, the source data model(s), and the corresponding edge(s) in the project's Driver–Pressure–State–Impact–Response (DPSIR) causal graph (`core/DAG.dot`). It is intended as a working reference while preparing a manuscript describing CrossTwin's indicator methodology, and as a running record of which causal relationships are backed by executable derivation logic versus flat, manually entered fields. Status tags (implemented / partial / gap) follow the coverage audit in `TODO.md`.

Contents

1 Scope and conventions **3**

2 Administrative app — population projection **3**

2.1 Compound annual growth rate — `extrapolate()` . . . 3
2.2 City-level growth factor — `_city_growth_factor()` . . . 4
2.3 District/Neighbourhood projection — `population_by_year()` . . . 4
2.4 Province projection . . . 4
2.5 What-if growth adjustment — `_adjust()` . . . 5
2.6 City annual growth rate — `annual_growth_rate()` . . . 5
2.7 Population and density cascade — `City.save()`, `Province.save()` . . . 5
2.8 Urban area cascade and source computation — `administrative/signals.py`, `physicalEnv/signals.py`
2.9 Urban-fabric buffer around new buildings — `urbanize_around()`, `release_ring()` . 6
2.10 Green area of new green spaces (parks) — `green_area()`, `release_green_area()` . . 7
2.11 Surface albedo — `physicalEnv/calculations/albedo.py` . . . 7
2.12 Building changes on the DSM — `dsm_patches.record_change()`, `apply_patches()` 8
2.13 Vegetation changes on the DSM — `dsm_patches.record_vegetation_change()` . . 8

3 Physical environment app — soil hydrology and infiltration **9**

3.1 Hydrologic soil group — `classify_soil_group()`, `SoilType.save()` . . . 9
3.2 Curve number — `curve_number()` . . . 10
3.3 Shallow groundwater and drained land — `soil_landcover_composition()` . . . 11
3.4 SCS runoff and infiltration coefficient — `scs_runoff_mm()`, `infiltration_coefficient()` 12
3.5 Area-weighted unit summary — `summarize_infiltration()` . . . 12

3.6 Stored infiltration rate — `weighted_infiltration_rate_mm_h()` 12
3.7 HILUCS land-use codes — `parse_hilucs_href()`, `hilucs_q()` 13

4 Built-up app — Degree of Urbanisation **13**
4.1 Allocation: people → residential properties → cells — `allocate()` 13
4.2 Cell density — `density()` 14
4.3 Grid classes — `classify()` 14
4.4 Unit class and urbanisation rate — `classify_unit()`, `urbanization_rate()` 14

5 Water supply app **14**
5.1 Per-capita consumption — `_get_consumption_capita()` 15
5.2 Total water demand — `calculate_total_demand()`, `TotalWaterDemand.save()` . . 15
5.3 Total consumption — `ConsumptionCapita.save()` 15
5.4 Source recharge — `AvailableFreshWater.save()` 15
5.5 Yearly recharge depth — `recharge_depth_cm_yr()` 15
5.6 Total extraction — `calculate_total_extraction()` 16
5.7 Total production — `calculate_total_production_day()` 16
5.8 Supply security & service time 16
5.9 Energy consumption — `calculate_energy_consumption()` 17
5.10 Pump emissions — `ExtractionWater.save()` 17
5.11 Extraction OPEX and environmental cost — `ExtractionWater.save()` 17
5.12 Consolidated OPEX — `OPEX.save()` 17
5.13 OPEX recovery (dashboard) — `calculate_opex_recovery()` 18
5.14 Billed consumption and revenue — `MeteredResidential.save()` 18
5.15 Collection ratio — `calculate_collection_ratio()` 18
5.16 Coverage — `calculate_coverage()`, `CoverageWaterSupply.save()` 18
5.17 Non-Revenue Water and ILI 19
5.18 Water quality — `calculate_water_quality()` 19
5.19 Available fresh water and drought area 19
5.20 Infiltration indicator (dashboard) — `calculate_infiltration()`, `_infiltration_indicators()` 20
5.21 Pipe geometry — `PipeNetwork.save()` 20

6 Housing app **20**
6.1 Supply/demand deficit 20
6.1.1 Household size and housing demand — `HouseholdSize.save()`, `HousingSupplyDemand.save()` 2
6.2 Pipeline units — `calculate_new_units()` 21
6.2.1 Pipeline land take — `calculate_pipeline_land_take()` 21
6.3 Mortgage indicators — `calculate_mortgage_indicators()` 22
6.4 House Price Index — `calculate_house_price_index()` 22
6.5 Affordability index — `HousingAffordability.save()` 22
6.6 Zoning composition — `calculate_zoning()` 22
6.7 Property market and vacancy — `calculate_property_indicators()` 23

7 Urban heat app **23**
7.1 Green area and vegetation coverage 23
7.2 Street canyon aspect ratio — `builtup.StreetCanyon`, `calculate_canyon_aspect()` 23
7.3 Raster statistics — `_raster_stats_sql()`, `get_thermal_indices()` 24
7.4 Thermal stress classification 24

7.5 Land surface temperature, Statistical Mono-Window — urban_heat/lst/ 25
7.6 Thermal comfort with SOLWEIG — urban_heat/solweig/ 26
7.7 Nature-Based Solutions coverage — calculate_nbs_coverage() 27

8 Map editing and what-if backup (mainMap app) 27
8.1 Editable-layer allowlist — EDITABLE_LAYERS, is_editable() 27
8.2 Attribute and geometry form — _build_form(), feature_form() 28
8.3 Identifiers for new objects — _next_negative_pk() 28
8.4 Deferred-constraint check — _check_constraints() 28
8.5 Change journal — journal() 29
8.6 Restore — restore_backup() 29
8.7 Map-side editor — Editing.js 29

9 Imported climate-effect indicators (importer app, Klimaateffectatlas) 30
9.1 Value transforms — _prepare_features() 30
9.2 Join and write rules — _keep_existing(), _enrich_rows() 30

10 Summary table 31

1 Scope and conventions

Each entry below documents one function or model `save()` method from the codebase. Entries are grouped by Django app, matching the project's domain decomposition. For every entry we record:

- **Location** — file and line number in the repository at time of writing.
- **Equation** — the exact arithmetic the code performs, in mathematical notation.
- **Variables** — symbol, meaning, unit, and source field/model.
- **DAG edge(s)** — the causal edge(s) in `core/DAG.dot` this computation is claimed to implement.
- **Status** — whether the DAG edge is genuinely derived from its source field (implemented), derived but with a known caveat or partial dependency (partial), or claimed in a docstring without being computed (gap).

Units follow the project convention: population counts are dimensionless integers; areas are km^2 unless noted; water volumes are m^3 or Mm^3 (explicitly distinguished per entry, since several models mix the two); monetary values are EUR.

2 Administrative app — population projection

Source: `administrative/calculations/population.py`, `administrative/models.py`. Population data originates from CBS table 85173NED (*Regionale prognose 2023–2050*), published per *gemeente* (city) [1].

2.1 Compound annual growth rate — `extrapolate()`

Location: `administrative/calculations/population.py:67`

Projects a city's population for a year outside the imported CBS horizon, using the compound annual growth rate implied by the earliest and latest imported years.

$$r = \left(\frac{v_1}{v_0}\right)^{\frac{1}{y_1 - y_0}} - 1 \tag{1}$$

$$P(y) = v_a \, (1 + r)^{\,(y - y_a)} \tag{2}$$

Variables:

- y_0, y_1 — earliest / latest year with an imported `PopulationProjection` record for the city
- v_0, v_1 — projected population at y_0, y_1
- r — implied compound annual growth rate
- y_a, v_a — anchor year/value: (y_1, v_1) if $y > y_1$, else (y_0, v_0)

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.2 City-level growth factor — `_city_growth_factor()`

Location: `administrative/calculations/population.py:103`

Ratio of a city's CBS-projected population between a baseline year and a target year, used to scale District/Neighbourhood populations without relying on possibly-incomplete local administrative coverage.

$$f(y) = \frac{P_{\text{city}}(y)}{P_{\text{city}}(y_{\text{baseline}})}, \qquad y_{\text{baseline}} = \begin{cases} 2025 & \text{if imported} \\ \min(y_{\text{imported}}) & \text{otherwise} \end{cases} \tag{3}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* (District/Neighbourhood level) — implemented

2.3 District/Neighbourhood projection — `population_by_year()`

Location: `administrative/calculations/population.py:124`

$$P_{\text{unit}}(y) = P_{\text{unit}}^{\text{current}} \times f(y) \tag{4}$$

Applied to the unit's own accurate current population rather than its share of `city.currentPopulation`, because the latter reflects only the districts/neighbourhoods imported locally, not CBS's whole-city total (see docstring, `administrative/calculations/population.py:11--22`).

Fallback (fewer than two imported years for the city): share-of-city approach,

$$P_{\text{unit}}(y) = \frac{P_{\text{unit}}^{\text{current}}}{P_{\text{city}}^{\text{current}}} \times P_{\text{city}}(y) \tag{5}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.4 Province projection

Location: `administrative/calculations/population.py:132--138`

$$P_{\text{province}}(y) = \sum_{c \,\in\, \text{cities of province}} P_c(y) \tag{6}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.5 What-if growth adjustment — `_adjust()`

Location: `administrative/calculations/population.py:59`

$$v_{\text{adjusted}} = v \times \left(1 + \frac{g}{100}\right)^{\max(y-2025,\ 0)} \tag{7}$$

where g is `growth_adjust_pct`, a user-supplied what-if slider bounded to $[-5, +5]$ (`population_params()`, `administrative/calculations/population.py:205`).

2.6 City annual growth rate — `annual_growth_rate()`

Location: `administrative/calculations/population.py:90` (as of the `City.popGrowthRate` implementation)

$$\texttt{popGrowthRate} = \left(\left(\frac{v_1}{v_0}\right)^{\frac{1}{y_1 - y_0}} - 1\right) \times 100 \tag{8}$$

Same CAGR as `extrapolate()`, expressed in percent per year and written onto `City.popGrowthRate` in `City.save()` (`administrative/models.py`) whenever at least two `PopulationProjection` years exist for the city.

DAG edge: *DAG edge: Population_Growth → Total_Population* (`City.popGrowthRate`) — implemented

2.7 Population and density cascade — `City.save()`, `Province.save()`

Location: `administrative/models.py`

$$P_{\text{city}} = \sum_{d \in \text{districts of city}} P_d \quad \text{(only if districts report a nonzero total)} \tag{9}$$

$$P_{\text{province}} = \sum_{c \in \text{cities of province}} P_c \tag{10}$$

$$\rho = \frac{P}{A}, \qquad A = \frac{\text{geom.area}}{10^6}\ \text{km}^2 \tag{11}$$

Density ρ in people/km^2. The Neighbourhood → District → City → Province cascade is signal-driven (`administrative/signals.py`), using `QuerySet.update()` rather than `.save()` to avoid re-triggering `post_save` recursively.

DAG edge: (population aggregation, not a numbered DAG node) — implemented

2.8 Urban area cascade and source computation — `administrative/signals.py`, `physicalEnv/signals.py`

Location: `administrative/signals.py` (cascade), `physicalEnv/signals.py` (source)

$$A^{\text{urban}}_{\text{nbhd}} = \sum_{v \in \text{qualifying LandCoverVector, latest year}} |\text{geom}_{\text{nbhd}} \cap \text{geom}_v| \,/10^6 \text{ km}^2 \tag{12}$$

$$A^{\text{urban}}_{\text{district}} = \sum_{n \in \text{neighbourhoods of district}} A^{\text{urban}}_n \tag{13}$$

$$A^{\text{urban}}_{\text{city}} = \sum_{d \in \text{districts of city}} A^{\text{urban}}_d \tag{14}$$

$$A^{\text{urban}}_{\text{province}} = \sum_{c \in \text{cities of province}} A^{\text{urban}}_c \tag{15}$$

A `physicalEnv.LandCoverVector` polygon qualifies if its `land_cover_type.class_name` case-insensitively contains any of *urban fabric*, *road*, *rail*, *transport*, restricted to the most recent `year` present for that city (older survey years are not summed, to avoid double-counting repeat surveys). The intersection area is computed in PostGIS via `ST_Intersection` (Django's `Intersection`/`Area` functions). Neighbourhood → District → City → Province propagation reuses the same `_recompute_population()` helper as the population cascade above, generalised to sum `urban_area` alongside `currentPopulation` in one aggregate; both use `QuerySet.update()`, not `.save()`. Unlike population, an aggregate with no children reporting a value is left `None` (unknown) rather than coerced to 0. Because a `LandCoverVector` write only fires `post_save` on itself (not on `Neighborhood`), the `physicalEnv` signal re-enters the `administrative` cascade manually after writing each affected Neighbourhood's `urban_area`.

DAG edge: *DAG edge: LandCover → CityArea* — implemented (added on 2026-11-10 when the `Urbanization` node was removed, `docs/DAG_REVISIONS.md`; the neighbourhood `urban_area` is the urban extent that `CityArea` stands for)

2.9 Urban-fabric buffer around new buildings — `urbanize_around()`, `release_ring()`

Source: `physicalEnv/calculations/landcover.py`, receiver in `physicalEnv/signals.py`. When a building is drawn in the map editor (negative id), the land cover of its city's latest year is rewritten around it:

$$B = \text{buffer}(\text{geom}_{\text{building}},\ 10 \text{ m}) \cap \text{geom}_{\text{city}} \tag{16}$$

$$\mathcal{L}' = \{\, P \setminus B : P \in \mathcal{L},\ P \setminus B \neq \emptyset \,\} \cup \{B \text{ as urban fabric}\} \tag{17}$$

$$\text{percentage}_P = |P|/10^6 \,/\, A_{\text{city}} \times 100 \tag{18}$$

where $\mathcal{L}$ is the set of `LandCoverVector` polygons of the city's latest `year`. The buffer is cut out of every polygon it touches (`ST_Difference`; polygons left empty are deleted) and inserted as one new polygon R_b of class *urban fabric*, tagged with the building id (`source_building`), so the layer stays free of overlaps and the urban-area cascade above counts each square metre once. Each piece $c_P = P \cap B$ is stored as an `UrbanFabricCut` with P's id and attributes.

Release (move or delete). Deleting the building (`building_deleted`) or moving it (`building_created` on a changed footprint, followed by a new buffer around the new footprint) runs `release_ring(b)`:

$$P' = P \cup (c_P \cap R_b) \quad \text{for every cut } c_P \text{ of } b \tag{19}$$

$$c'_{b',T} = c_{b',R_b} \cap c_{b,T} \quad \text{for every cut a later ring } b' \text{ took from } R_b \tag{20}$$

Only the area R_b still holds is given back, so what a later ring b' took from it stays urban; P is recreated with its original id and attributes when it was swallowed whole. The second line re-targets

b''s cuts onto the polygons that lay under R_b, so releasing b' later returns its land to them rather than to the deleted ring. Then R_b and its cuts are deleted and `percentage` is recomputed for the rows given land back. Rings, cuts and recreated rows are journalled like the buffer itself; during a backup restore (`core.editing.is_restoring()`) the delete receiver is skipped, since the restore puts the rows back itself. Imported (BAG, positive id) buildings never trigger either step; rings created before `source_building` existed cannot be released.

DAG edge: *DAG edge: Buildings → LandCover* — implemented (cited as `New_Units → LandCover` before 2026-11-10; stored for map-editor buildings; the housing pipeline's land take is a dashboard statistic, §6.2.1)

2.10 Green area of new green spaces (parks) — `green_area()`, `release_green_area()`

A green space k (`nature.GreenSpaces`; a park is a green space of `type park`, there is no separate park model since 2026-11-11) drawn or reshaped in the map editor (`green_space_saved`, only inside `core.editing.editor_change()`) turns its outline, clipped to the city, into green urban area with the same cut / give-back mechanics as the building ring, without a buffer:

$$G_k = \text{park}_k \cap \text{city} \quad \mathcal{L}' = \{\, P \setminus G_k : P \in \mathcal{L},\ P \setminus G_k \neq \emptyset \,\} \cup \{G_k \text{ as green urban areas}\} \tag{21}$$

The new polygon has class *Green urban areas* (CORINE 1.4.1 [23]), is tagged with `source_park` (the green space's id; the column name is kept from the former `builtup.Park`), and takes the material *Grass*, or *Deciduous trees* when the park's `vegetationType` mentions trees. Its cut pieces are `UrbanFabricCut` rows with `owner_park` (exactly one of `owner_building`/`owner_park` is set), so a park and a building ring overlapping each other release in either order. Moving or deleting the park runs `release_green_area(k)`, the release equations above with $R_k = G_k$. Imported green spaces (OpenStreetMap parks, BGT) never trigger either step. Downstream, the change reaches the urban-area cascade, the stored recharge of `AvailableFreshWater` and the SOLWEIG land-cover grid like any other land-cover edit.

DAG edge: *DAG edge: New_Green_Area → LandCover* — implemented (map-editor green spaces; a separate Response node, since *DAG edge: LandCover → Green_Area* already exists and the reverse edge would close a cycle)

2.11 Surface albedo — `physicalEnv/calculations/albedo.py`

Every land-cover polygon has an effective albedo

$$\alpha_P = \begin{cases} \alpha_P^{\text{meas}} & \text{measured or estimated albedo stored on } P \\ \alpha(\text{material}_P) & \text{otherwise} \end{cases} \tag{22}$$

where material_P defaults to the class's `default_material`, chosen by keywords in the class name (water, trees, cropland, grass, sand, bare soil, asphalt for roads and rail, mixed urban fabric). The materials are Table 5.1 of Oke et al. [22] (45 natural and urban surfaces with their albedo range); a material's typical albedo is the table's average where it gives one (urban 0.14, suburban 0.15), else the midpoint of the range (e.g. asphalt 0.16, grass 0.21, deciduous forest 0.165, water at high sun 0.065). The material fields of land cover, roofs and streets offer only the categories that fit them.

Measured. A Sentinel-2 L2A composite (`sentinel2_albedo_raster`, openEO) gives the shortwave broadband albedo by narrow-to-broadband conversion [20]:

$$\alpha = 0.2266\,\rho_{B2} + 0.1236\,\rho_{B3} + 0.1573\,\rho_{B4} + 0.3417\,\rho_{B8} + 0.1170\,\rho_{B11} + 0.0338\,\rho_{B12} \tag{23}$$

with ρ the surface reflectance (DN / 10 000). Saving the raster (`albedo_imagery_saved`) stores the zonal mean $\alpha_P^{\text{meas}} = \overline{\alpha(x)}_{x\in P}$ (`ST_SummaryStats(ST_Clip(...))`) on every latest-year polygon it covers with at least 3 valid pixels and $0 \le \alpha \le 1$, skipping editor polygons and polygons measured on a newer scene. Bulk land-cover imports measure their new polygons against albedo rasters imported earlier. A geometry change drops the measured value.

Estimated (editor building ring). With footprint F and ring R_b: $\alpha_{R_b} = \big(|F|\,\alpha_{\text{roof}} + (|R_b| - |F|)\,\alpha_{\text{paving}}\big)/|R_b|$, the roof being the building's `roofMaterial` or *tar and gravel, bitumen* (0.13), the paving *concrete* road (0.225).

DAG edge: *DAG edge: RS_Imagery → LandCover* — partial (albedo attribute of the land cover; the classification itself still comes from the land-cover import). No model reads the albedo yet.

2.12 Building changes on the DSM — `dsm_patches.record_change()`, `apply_patches()`

Source: `physicalEnv/dsm_patches.py`, receivers `building_dsm_saved/building_dsm_deleted` in `physicalEnv/signals.py`. A building created, reshaped, re-heighted or deleted in the map editor (`core.editing.is_editor_change()`; imports never) is stored as one `DSMBuildingPatch` row: its footprint before the first edit F_0 (none for a drawn building), its current footprint F (none when deleted) and its height above ground

$$h = \begin{cases} \texttt{height_m} & \text{if set} \\ 3\,\text{m} \times \texttt{numberFloors} & \text{if set} \\ \text{none} & \text{otherwise (only } F_0 \text{ is cleared)} \end{cases} \tag{24}$$

The DSM COG is never rewritten. Every read of a DSM window for analysis (`svf.load_dsm`: the SVF tool and the SOLWEIG tool) applies the patches that touch it, on the window's grid:

$$z'(x) = g(x) \qquad x \in \bigcup F_0 \tag{25}$$

$$z'(x) = \text{median}_{y\in F}\, g(y) + h \qquad x \in F \quad \text{(flat roof, applied after the clearing)} \tag{26}$$

$$z'(x) = z(x) \qquad \text{otherwise} \tag{27}$$

where z is the DSM and g the ground: the DEM read on the same grid, or without a DEM the minimum filter of z over a 40 m window (`estimate_ground`). Cells are assigned by their centre (`rasterio.features.rasterize`). Patches are journalled for the editor's backup (`dsm_patch_rewriting/dsm_patc`) so Restore reverts them with the buildings. Every change of a patch, Restore included, appends a `DSMEdit` with the bounding box of the old and new footprints; an SVF raster or a SOLWEIG run older than an overlapping `DSMEdit` is flagged as stale on the map (`urban_heat/solweig/pipeline.py::stale_resu`

DAG edge: *DAG edge: Buildings → DSM* — implemented (editor changes as an overlay on the DSM; flat roofs)

2.13 Vegetation changes on the DSM — `dsm_patches.record_vegetation_change()`

Source: `physicalEnv/dsm_patches.py`, receivers `tree_dsm_saved/tree_dsm_deleted` and `forest_dsm_saved/fore` in `physicalEnv/signals.py`. A `nature.Tree` or `nature.Forests` row created, moved, resized or deleted in the map editor (imports never: AHN already measured it) is stored as one `DSMVegetationPatch`: the geometry before the first edit V_0, the current one V and a canopy height c. Parks and green spaces are not sources (their outlines do not record trees). A tree's geometry is its crown disc of radius

$$R = \tfrac{1}{2}\begin{cases} \texttt{crownDiameter_m} & \text{if set} \\ 0.6 \times h_t & \text{otherwise} \end{cases}, \qquad c = h_t = \begin{cases} \texttt{height_m} & \text{if set} \\ 10\,\text{m} & \text{otherwise} \end{cases} \tag{28}$$

and a forest's is its polygon with $c = 15\,\mathrm{m}$. With B the stored building footprints and $\tau = 2.5\,\mathrm{m}$ (also SOLWEIG's tree threshold), the overlay is applied after the building originals are cleared and before the current buildings are raised:

$$z'(x) = g(x) \qquad x \in C,\ z(x) - g(x) > \tau,\ x \notin B \tag{29}$$

$$z'(x) = \max\big(z(x),\ g(x) + c \cdot \max(1 - r^2/R^2,\ 0.5)\big) \qquad x \in V \quad \text{(tree)} \tag{30}$$

$$z'(x) = \max\big(z(x),\ g(x) + c\big) \qquad x \in V \setminus V_0 \quad \text{(forest)} \tag{31}$$

with $C = V_0$ for a tree (its whole old crown) and $C = V_0 \setminus V$ for a forest (only the area it gave up; the area it kept keeps the measured canopy), and r the distance of the cell centre to the crown's centre. A new roof is applied after the crown, so it always wins. Patches are journalled like building patches and append a `DSMEdit`, so SVF and SOLWEIG results over them are flagged as stale.

DAG edge: *DAG edge: Vegetation_Coverage → DSM* — implemented (editor trees and forests as an overlay on the DSM)

3 Physical environment app — soil hydrology and infiltration

Source: `physicalEnv/calculations/soil.py`, `physicalEnv/hilucs.py`, `physicalEnv/models.py`. Soil polygons come from the Dutch soil map 1:50 000 (*Bodemkaart*, 2025 edition) and groundwater depths from the groundwater-level grids of BIS Nederland [19]; land cover is the CORINE-classified BRT *Bodemgebruik* layer [23]. Together they implement the `LandCover` → `Infiltration` and `Soil_Type` → `Infiltration` edges with the SCS Curve Number method [16, 17, 18]. Everything except the spatial composition query (§3.3) is pure Python, so the dashboard re-evaluates the equations for any rain depth without querying the database.

3.1 Hydrologic soil group — `classify_soil_group()`, `SoilType.save()`

Location: `physicalEnv/calculations/soil.py:82`, `physicalEnv/models.py:61`

The soil map names each unit in Dutch with the texture of its layers (e.g. *Koopveengronden op zand*: peat on sand). Each texture word w found in the name n maps to a hydrologic soil group $g(w)$, and the most restrictive layer sets the group, since the least transmissive layer limits infiltration [17]:

$$G(n) = \max_{\prec}\{\, g(w) : w \in \mathrm{words}(n) \,\}, \qquad \mathrm{A} \prec \mathrm{B} \prec \mathrm{C} \prec \mathrm{D} \tag{32}$$

with $G(n)$ undefined when the name contains no texture word. The phrase *geen zand* ("no sand") is removed before matching.

Group	Dutch texture words (soil map)	Texture
A	*zand*, *grind* (incl. loamy and clayey sand)	sand, gravel
B	*zavel* (8–25 % lutum), *leem* (not *leemarm*, *lemig*, *keileem*)	loam, silt loam
C	*lichte klei*; *knip* (dense knip-clay horizon)	clay loam
D	*klei*, *zware klei*, *veen*, *moerig*, *slik*, *drecht/lied* (clay on peat), *keileem*, *potklei*, *kleefaarde*	clay, peat

All 171 distinct soil names of the 2025 map receive a group (A: 46 names, B: 33, C: 16, D: 76). `SoilType.save()` stores $G(n)$ and the final infiltration rate range $[f_{\min}, f_{\max}]$ of the group [17]:

Group	Final infiltration rate f (mm/h)	Runoff potential
A	> 7.6	low
B	3.8–7.6	moderate
C	1.3–3.8	moderately high
D	0–1.3	high

DAG edge: *DAG edge: Soil_Type → Infiltration* — implemented (together with §3.3–3.5)

3.2 Curve number — `curve_number()`

Location: `physicalEnv/calculations/soil.py:138`

The curve number $\mathrm{CN}(c, G)$ of a land-cover class c on soil group G is read from the TR-55 tables for antecedent moisture condition II [16]. CORINE class names are matched by keyword at a word start (so *road* does not match *broad-leaved forest*); the first matching cover type wins:

CORINE class (keyword)	TR-55 cover type	A	B	C	D
water courses/bodies, sea, lagoons, estuaries	open water	100	100	100	100
marshes, peat bogs, salines, intertidal flats	saturated wetland (impervious)	98	98	98	98
green urban areas, sport and leisure	open space, good condition	39	61	74	80
discontinuous urban fabric; urban fabric	residential, $\leq 1/8$ acre (65 % imperv.)	77	85	90	92
continuous urban fabric	commercial and business (85 % imperv.)	89	92	94	95
industrial/commercial units, ports, airports	industrial (72 % imperv.)	81	88	91	93
road and rail networks	paved roads, open ditches, incl. right-of-way	83	89	92	93
mineral extraction, dumps, construction	newly graded areas	77	86	91	94
arable land, irrigated land, rice	row crops, straight row, good	67	78	85	89
vineyards, fruit trees, olive groves	woods–grass combination (orchard), good	32	58	72	79
natural grasslands	meadow	30	58	71	78
pastures	pasture, good	39	61	74	80
complex cultivation, agro-forestry	small grain, straight row, good	63	75	83	87
forests (broad-leaved, coniferous, mixed)	woods, good	30	55	70	77
moors and heathland	brush, good	30	48	65	73
sclerophyllous vegetation, transitional woodland	brush, fair	35	56	70	77
beaches, dunes, bare rock, sparse vegetation, burnt	fallow, bare soil	77	86	91	94

Imperviousness of built-up land enters through these curve numbers; a class without a match (e.g. glaciers) or a piece without a soil group is left out and reported as unclassified.

3.3 Shallow groundwater and drained land — `soil_landcover_composition()`

Location: `physicalEnv/calculations/soil.py:206`; constants `SHALLOW_GROUNDWATER_CM` (`:30`), `SEASONS` (`:34`), `DRAINED_LAND_COVER` (`:48`)

A soil whose water table lies within $h^* = 60\,\text{cm}$ of the surface belongs to group D whatever its texture; NEH 630 gives such soils a dual group (A/D, B/D, C/D), the texture letter applying where the land is drained [17]. The query intersects the unit u with the soil map and the most recent land-cover year, and splits every piece p by the area where the season's groundwater depth h_s is

shallower than h^*:

$$S_s = \bigcup \{ \text{ cells } x \subset u : h_s(x) < h^* \} \tag{33}$$

$$G'(p) = \begin{cases} \mathrm{D} & \text{on } p \cap S_s \text{ if } c(p) \notin \mathcal{D} \\ G(p) & \text{otherwise} \end{cases} \tag{34}$$

where $s \in \{\text{wet}, \text{dry}\}$ selects $h_{\text{wet}} = \text{GHG}$ (mean highest groundwater level) or $h_{\text{dry}} = \text{GLG}$ (mean lowest), and $\mathcal{D}$ is the set of drained land covers (urban fabric, industrial/commercial units, road and rail networks, port areas, airports). GHG and GLG are 50 m grids in cm below ground level, capped at 254 ("254 cm or deeper"); cells without a value (255) are not shallow. BIS Nederland leaves most built-up cells (85 % in Enschede) and water bodies without a value, consistent with drained, impervious land; water and wetlands need no rule since their curve number is independent of the soil group. The cells are reclassified to shallow/deep and dissolved into polygons (`ST_Reclass`, `ST_DumpAsPolygons`, `ST_Union`) before the intersection, so each piece is split once rather than per cell. The function returns the area of every (G', c) combination, the texture-only group areas, $|S_s \cap \bigcup_{c \notin \mathcal{D}} p|$ (moved to D) and $|S_s \cap \bigcup_{c \in \mathcal{D}} p|$ (kept as drained).

3.4 SCS runoff and infiltration coefficient — `scs_runoff_mm()`, `infiltration_coefficient()`

Location: `physicalEnv/calculations/soil.py:149, 165`

$$S = \frac{25400}{\mathrm{CN}} - 254, \qquad I_a = 0.2\,S \tag{35}$$

$$Q = \begin{cases} \dfrac{(P - I_a)^2}{P - I_a + S} & P > I_a \\ 0 & P \le I_a \end{cases} \tag{36}$$

$$C_{\text{inf}} = \frac{P - Q}{P} \tag{37}$$

where P is the rain depth of one event (mm), S the potential maximum retention (mm), I_a the initial abstraction (mm), Q the direct runoff (mm) and C_{inf} the infiltration coefficient, the share of the rain that does not run off [18]. $\mathrm{CN} = 100$ gives $Q = P$.

3.5 Area-weighted unit summary — `summarize_infiltration()`

Location: `physicalEnv/calculations/soil.py:172`

For the pieces i of a unit, with area A_i and curve number $\mathrm{CN}_i = \mathrm{CN}(c_i, G'_i)$:

$$V_{\text{inf}} = \sum_i A_i \, \frac{P - Q(P, \mathrm{CN}_i)}{1000}, \qquad V_{\text{run}} = \sum_i A_i \, \frac{Q(P, \mathrm{CN}_i)}{1000} \quad [\mathrm{m}^3] \tag{38}$$

$$C_{\text{inf},u} = \frac{V_{\text{inf}}}{V_{\text{inf}} + V_{\text{run}}}, \qquad \overline{\mathrm{CN}} = \frac{\sum_i A_i \, \mathrm{CN}_i}{\sum_i A_i} \tag{39}$$

Runoff is computed per piece before weighting: since Q is convex in CN, evaluating $Q(P, \overline{\mathrm{CN}})$ would overstate infiltration. $\overline{\mathrm{CN}}$ is reported for reference only.

3.6 Stored infiltration rate — `weighted_infiltration_rate_mm_h()`

Location: `physicalEnv/calculations/soil.py::weighted_infiltration_rate_mm_h`, called by `AvailableFreshWater.save()` (`watersupply/models.py`)

For the pieces i of a fresh-water source polygon (wet-season groundwater, so G'_i includes the shallow-groundwater rule), with $\bar{f}(G)$ the middle of the group's final rate range (group A: $f_{\min} = 7.6\,\text{mm/h}$, it has no upper bound) and $s(c)$ the sealed share of land-cover class c:

$$f_{\text{src}} = \frac{\sum_i A_i\, \bar{f}(G'_i)\,\big(1 - s(c_i)\big)}{\sum_i A_i}\ [\text{mm/h}], \qquad \texttt{infiltrationRate_cm_h} = f_{\text{src}}/10 \tag{40}$$

$s(c)$: residential (discontinuous) urban fabric 0.65, continuous urban fabric 0.85, industrial/commercial 0.72 (TR-55 table 2-2a [16]); open water and wetlands 1 (their CN turns all rain into runoff); roads and rail 0.6 (estimate: TR-55 gives no share for roads with open ditches); every other class 0. Without soil or land-cover pieces the stored value is kept.

DAG edges: *DAG edge: LandCover → Infiltration*, *DAG edge: Soil_Type → Infiltration* — implemented (stored; refreshed by `watersupply/signals.py` when `SoilArea`, `SoilType`, `LandCoverVector` or `GroundwaterDepth` change)

3.7 HILUCS land-use codes — `parse_hilucs_href()`, `hilucs_q()`

Location: `physicalEnv/hilucs.py:18, 48`

Zoning elements of the INSPIRE Planned Land Use feed reference their land use as a URI of the HILUCS codelist [24], e.g. `.../HILUCSValue/6_3_2_WaterAreasNotInOtherEconomicUse`. The code is read from the URI's leading digits (`6_3_2` ↦ 6.3.2) and matched against `HILUCSLandUse.code` (98 registry values, loaded by migration); the full URI is never used, since the PDOK file misspells some (`5_2_...Comptible...`). A code k selects itself and every code below it in the hierarchy:

$$\text{sel}(k) = \{\, k' : k' = k \ \vee\ k' \text{ starts with } k. \,\} \tag{41}$$

4 Built-up app — Degree of Urbanisation

Source: `builtup/calculations/degurba.py` (grid rules), `builtup/calculations/urbanisation.py` (allocation, refresh, what-if), `builtup/calculations/chain.py` (unit class, rate), `builtup.Urbanisation.save()` Method, deviations and decision log: `docs/DEGREE_OF_URBANISATION.md`. Replaces the former `Urbanization` node (`docs/DAG_REVISIONS.md`).

4.1 Allocation: people → residential properties → cells — `allocate()`

$$p_d = \frac{P_n}{N_n} \qquad d \in \text{residential properties of neighbourhood } n \tag{42}$$

$$P_c = \sum_{d \in c} p_d \qquad \text{or, if } N_n = 0: \quad P_c = P_n \frac{|n \cap c|}{|n|} \tag{43}$$

$$B_c = \sum_b |b \cap c| \qquad \text{building footprints, } b \text{ assigned to } n \text{ by point on surface} \tag{44}$$

P_n = `Neighborhood.currentPopulation`, N_n = `builtup.Property` rows with *Residential function*; cells c of r = 100 m and 1000 m aligned to the EPSG:28992 origin (CBS square grid). Settlement share s_c and water share w_c from the latest `LandCoverVector` year; a cell is settlement land if $s_c \geq 0.5$ or $P_c > 0$.

4.2 Cell density — `density()`

$$\rho_c^{1\,\mathrm{km}} = \frac{P_c}{r^2(1-w_c)} \tag{45}$$

$$\rho_c^{100\,\mathrm{m}} = \begin{cases} \dfrac{\sum_{k\in W_3(c)} P_k}{\sum_{k\in W_3(c),\ k\ \text{settlement}} r^2} & c \text{ settlement land} \\ \dfrac{\sum_{k\in W_3(c)} P_k}{\sum_{k\in W_3(c)} r^2(1-w_k)} & \text{otherwise} \end{cases} \tag{46}$$

$W_3(c)$ = the 3×3 window around c (300 m). Built-up share $\beta_c = \sum_W B_k / \sum_W r^2$ (window 1 at 1 km, 3 at 100 m). Worked example: three dwelling cells of 60 people and one street cell in a window of five fields give $\rho^{100\,\mathrm{m}} = 180/0.04 = 4\,500$ inh./km^2, against 2 000 over the whole window and about 800 over a 1 km window.

4.3 Grid classes — `classify()`

Urban centre (30): components of $\{\rho \geq 1500 \vee \beta \geq 0.5\}$ (4-connected; settlement cells only at 100 m) with $\sum P \geq 50\,000$; holes < 15 km^2 filled (at 100 m only holes made of settlement cells); cells with ≥ 5 of their 8 neighbours in the centre added until stable (settlement cells only at 100 m). Dense urban cluster (23): the same cells, 4-connected, $5\,000 \leq \sum P < 50\,000$, not in a centre. Urban cluster: $\rho \geq 300$, 8-connected, $\sum P \geq 5\,000$; semi-dense (22) if it has no centre or dense cell and lies > 2 km from one, otherwise its remaining cells are suburban (21). Rural cluster (13): $\rho \geq 300$, 8-connected, $500 \leq \sum P < 5\,000$. Otherwise 12 ($\rho \geq 50$), 10 ($w \geq 0.5$, $P = 0$) or 11.

4.4 Unit class and urbanisation rate — `classify_unit()`, `urbanization_rate()`

$$L1 = \begin{cases} 3\ (\text{city}) & P_{30}/P \geq 0.5 \\ 2\ (\text{town}) & (P_{30}+P_{23}+P_{22}+P_{21})/P \geq 0.5 \\ 1\ (\text{rural}) & \text{otherwise} \end{cases} \tag{47}$$

$$\text{rate} = \left(\frac{A_{t_1}}{A_{t_0}}\right)^{1/(t_1-t_0)} - 1, \qquad A = \sum_{c:\ L2_c \in \{21,22,23,30\}} |\text{unit} \cap c| \tag{48}$$

P_k from the unit's allocation rows; t_0 the earliest stored CrossTwin year (GHSL epochs are reference layers only). `City.urbanizationRate` mirrors the 100 m city rate. What-if (dashboard): every allocation scaled by $f = P(\text{city}, y, \text{scenario}, g)/P(\text{city}, y_0, \text{median}, 0)$ and the grid reclassified in memory, without writing.

DAG edges: *DAG edge: Total_Population → Degree_Urbanisation*, *DAG edge: Buildings → Degree_Urbanisation*, *DAG edge: LandCover → Degree_Urbanisation* — implemented (receivers in `builtup/signals.py`, deferred by `deferred_urbanisation_refresh`)

5 Water supply app

Source: `watersupply/calculations/indicators.py`, `watersupply/models.py`, `watersupply/signals.py`.

5.1 Per-capita consumption — `_get_consumption_capita()`

Location: `watersupply/calculations/indicators.py:16`

$$\bar{c} = \text{avg}_{\,c \in \text{cities}(u)} \big(\texttt{consumption_capita_L_d}\big), \qquad [\bar{c}] = \text{L/person/day} \tag{49}$$

5.2 Total water demand — `calculate_total_demand()`, `TotalWaterDemand.save()`

Location: `watersupply/calculations/indicators.py:27`; `watersupply/models.py:70`

$$D_{\text{day}} = \frac{\bar{c}}{1000} \times P \qquad [\text{m}^3/\text{day}] \tag{50}$$

$$\texttt{demandDay} = \frac{P \times \bar{c}}{10^9} \qquad [\text{Mm}^3/\text{day}] \tag{51}$$

$$\texttt{demandYR} = \texttt{demandDay} \times 365 \tag{52}$$

where P is `city.currentPopulation`. Note the two representations differ by a factor of 10^6 (m^3 vs. Mm^3); `calculate_supply_security()` converts explicitly (§5.8).

DAG edges: *DAG edge: Total_Population → Total_Water_Demand*, *DAG edge: Consumption-Capita → Total_Water_Demand* — implemented

5.3 Total consumption — `ConsumptionCapita.save()`

Location: `watersupply/models.py:38`

$$\texttt{total_consumption_m3_yr} = \frac{\texttt{consumption_capita_L_d}}{1000} \times P \times 365 \tag{53}$$

DAG edges: *DAG edge: Total_Population → Total_Consumption*, *DAG edge: ConsumptionCapita → Total_Consumption* — implemented

5.4 Source recharge — `AvailableFreshWater.save()`

Location: `watersupply/calculations/chain.py::recharge_Mm3_yr`, called by `AvailableFreshWater.save()` (`watersupply/models.py`)

$$Y = \frac{d_{\text{inf}}}{100} \times \frac{A}{10^6}, \qquad [d_{\text{inf}}] = \text{cm/yr}\ (\texttt{infiltrationDepth_cm}),\ [A] = \text{m}^2,\ [Y] = \text{Mm}^3/\text{yr}\ (\texttt{yield_Mm3_year}) \tag{54}$$

A is the source polygon's area in the storage CRS. Recalculated on every save when d_{inf} is known; otherwise a supplied yield is kept. `infiltrationRate_cm_h` is a capacity during rain, not a yearly amount, and is not used.

5.5 Yearly recharge depth — `recharge_depth_cm_yr()`

Location: `watersupply/calculations/chain.py::recharge_depth_cm_yr`, called by `AvailableFreshWater.sa` meteorology from `weather/water_balance.py::yearly_water_balance`, imported by `weather/knmi.py` (catalog `knmi_daily_water_balance`)

With P_d the daily precipitation (KNMI `RH`) and E the year's Makkink reference evapotranspiration (sum of KNMI `EV24`) at the nearest station with a complete year (≥ 360 days; `recharge_year`, else its latest), and the pieces i of the source as in §3.6:

$$d_{\text{inf}} = \frac{1}{10} \max\Big(0,\ \sum_d P_d - \sum_d \sum_i \frac{A_i}{\sum_j A_j}\, Q(P_d, \text{CN}_i) - E\Big) \ [\text{cm/yr}] \tag{55}$$

Q per piece and day as in §3.5. Simplifications: crop factor 1 for every surface, no soil-moisture store between days (a dry year gives 0), pieces without a CN are left out. Without pieces or a complete year the stored d_{inf} is kept. Y then follows from d_{inf} as above.

DAG edges: *DAG edge: Infiltration → Available_FW*, *DAG edge: Meteorology → Available_FW* — implemented (refreshed by `watersupply/signals.py` on daily `Meteorology` rows and on `weather.knmi.daily_water_balance_changed`)

5.6 Total extraction — `calculate_total_extraction()`

Location: `watersupply/calculations/indicators.py` (`calculate_total_extraction`, `watershed_share`)

A well's water belongs to its watershed $h(w)$ (the `watershed` of its `AvailableFreshWater` source, else the watershed containing the well) and is shared by the population of that watershed, wherever the well lies. Neighborhoods are assigned to one watershed by their point on surface:

$$E_u = \sum_h s_{u,h} \sum_{w:\, h(w)=h} V_w \;+\; \sum_{w:\, h(w)=\varnothing,\ w \cap \text{geom}(u)} V_w \tag{56}$$

$$s_{u,h} = \frac{\sum_{n \in u,\ \text{pos}(n) \in h} P_n}{\sum_{\text{pos}(n) \in h} P_n} \quad \text{(area share } |u \cap h|/|h| \text{ when the watershed has no neighborhood population)} \tag{57}$$

V_w (`ExtractionWater.volume_m3_d()`, also used by `TotalWaterProduction`) is the first known of `current_extraction_Mm3_yr` $\times 10^6/365$, $q_w \times t_w \times 3600$ ($t_w =$ `OperationTime_h_day`) and $q_w \times 86400$, with $[q_w] = \text{m}^3/\text{s}$; $[V_w] = [E_u] = \text{m}^3/\text{day}$; $\sum_u s_{u,h} = 1$ over the neighborhoods of a watershed. Wells without a watershed keep the location rule.

DAG edge: *DAG edge: Available_FW → Total_Extraction* — implemented; the share also makes *DAG edge: Total_Population* of every city in the watershed an input of its neighbours' supply security (receivers `city_population_cascaded`, `city_saved`, `watershed_changed`, `extraction_changed` in `watersupply/signals.py`).

5.7 Total production — `calculate_total_production_day()`

Location: `watersupply/calculations/indicators.py:51`

$$\Pi = E + I, \qquad I = \sum_{\text{active imports}} \texttt{quantity_m3_d} \tag{58}$$

DAG edges: *DAG edge: Total_Extraction → Total_Water_Prod*, *DAG edge: Imported_Water → Total_Water_Prod* — implemented

5.8 Supply security & service time

Location: `calculate_supply_security()` (`watersupply/calculations/indicators.py:73`); `SupplySecurity.` (`watersupply/models.py:107`)

$$D_{\text{m}^3/\text{day}} = \texttt{demandDay} \times 10^6 \tag{59}$$

$$S = \frac{\Pi}{D_{\text{m}^3/\text{day}}} \qquad \text{(ratio; } \times 100 \text{ for \%)} \tag{60}$$

$$T_{\text{service}} = \begin{cases} 24 & \Pi \geq D_{\text{m}^3/\text{day}} \\ 24 \times \dfrac{\Pi}{D_{\text{m}^3/\text{day}}} & \Pi < D_{\text{m}^3/\text{day}} \end{cases} \qquad [\text{h/day}] \tag{61}$$

DAG edges: *DAG edge: Total_Water_Demand → Supply_Security, DAG edge: Total_Water_Prod → Supply_Security, DAG edge: Supply_Security → Service_Time* — implemented

5.9 Energy consumption — `calculate_energy_consumption()`

Location: `watersupply/calculations/indicators.py:100`

$$K = \sum_w \left(\texttt{pumpEnergyRate_kWh_h}_w \times \texttt{OperationTime_h_day}_w\right) + 1000 \times \sum \texttt{EnergyConsumption_MW_day} \tag{62}$$

$[K] = \text{kWh/day}$. **DAG edges:** *DAG edge: Total_Extraction → Energy_Consumption, DAG edge: Total_Water_Prod → Energy_Consumption, DAG edge: Energy_Cost → Energy_Consumption* — partial (Energy_Cost side of the edge is not read back into this function)

5.10 Pump emissions — `ExtractionWater.save()`

Location: `watersupply/models.py:317`

$$R_{\mathrm{CO_2}} = \texttt{pumpEnergyRate_kWh_h} \times \texttt{pumpEmmissionFactor_kg_CO2_kWh} \tag{63}$$

$$M_{\text{day}} = R_{\mathrm{CO_2}} \times \texttt{OperationTime_h_day}, \qquad M_{\text{yr}} = M_{\text{day}} \times 365 \tag{64}$$

DAG edge: *DAG edge: Total_Extraction → CO2_Emission* — implemented (per-well; `calculate_co2_emission()` sums across wells)

5.11 Extraction OPEX and environmental cost — `ExtractionWater.save()`

Location: `watersupply/models.py:346--357`

$$\texttt{opex_EUR_m3} = \texttt{labor} + \texttt{energy} + \texttt{chemicals} + \texttt{tax} \quad \text{(only when at least one component is known)} \tag{65}$$

$$\mathrm{CO_2\ cost} = \frac{M_{\text{day}}}{V_{\text{day}}} \times \text{price}_{\mathrm{CO_2}}, \qquad V_{\text{day}} = q \times \texttt{OperationTime_h_day} \times 3600 \tag{66}$$

5.12 Consolidated OPEX — `OPEX.save()`

Location: `watersupply/models.py:640`

$$\text{Cost}_{\text{extraction}} = \sum \left(\texttt{opex_EUR_m3} \times \texttt{current_extraction_Mm3_yr} \times 10^6\right) \tag{67}$$

$$\text{Cost}_{\text{imported}} = \sum \left(\texttt{quantity_m3_d} \times 365 \times \texttt{price_EUR_m3}\right) \tag{68}$$

$$\text{Cost}_{\text{treatment}} = \overline{\texttt{UnitaryOPEX_EUR_m3}}_{\text{year}} \times V_{\text{yr}} \tag{69}$$

$$\text{Cost}_{\text{network}} = \sum \left(\texttt{maitenanceCost_EUR_km} \times \texttt{length_km}\right) \tag{70}$$

$$\texttt{totalOPEX_EUR} = \text{Cost}_{\text{extraction}} + \text{Cost}_{\text{imported}} + \text{Cost}_{\text{treatment}} + \text{Cost}_{\text{network}} \tag{71}$$

$$\texttt{UnitaryOPEX_EUR_m3} = \frac{\texttt{totalOPEX_EUR}}{V_{\text{yr}}}, \qquad V_{\text{yr}} = V_{\text{extraction}} + V_{\text{imported}} \tag{72}$$

`TotalWaterProduction`'s own energy cost is deliberately excluded, since it is already embedded in `opex_EUR_m3` (see inline comment, `watersupply/models.py`).

$$\texttt{OPEX_recovered_PCT} = \frac{\sum \texttt{Recovery_EUR}}{\texttt{totalOPEX_EUR}} \times 100 \tag{73}$$

DAG edges: *DAG edge: Total_Extraction/Total_Water_Prod → OPEX*, *DAG edge: Imported_Water → OPEX*, *DAG edge: WT_Cost → OPEX*, *DAG edge: Network → OPEX*, *DAG edge: Metered_Res_Water → OPEX_Recovery* — implemented

5.13 OPEX recovery (dashboard) — `calculate_opex_recovery()`

Location: `watersupply/calculations/indicators.py:205`

$$\text{Recovery}\% = \frac{\sum_{\text{metered users}} \texttt{Recovery_EUR}}{\texttt{totalOPEX_EUR}_{\text{year}}} \times 100 \tag{74}$$

DAG edges: *DAG edge: OPEX → OPEX_Recovery*, *DAG edge: CollectionRatio → OPEX_Recovery*, *DAG edge: NRW → OPEX_Recovery* — partial (computes directly from stored totals; does not call `calculate_collection_ratio()` or reference NRW, despite the docstring)

5.14 Billed consumption and revenue — `MeteredResidential.save()`

Location: `watersupply/models.py:172`

$$\gamma = \frac{\texttt{collected_meters}}{\texttt{installed_meters}} \tag{75}$$

$$V_{\text{yr}} = \frac{\texttt{consumption_capita_L_d}}{1000} \times 365 \times \texttt{populationServed} \times \gamma \tag{76}$$

$$\texttt{Recovery_EUR} = V_{\text{yr}} \times \texttt{userTariff_EUR_m3} \tag{77}$$

Uses the city's most recent `ConsumptionCapita` record (not filtered by year), applied to local `populationServed` rather than the whole-city `total_consumption_m3_yr`.

5.15 Collection ratio — `calculate_collection_ratio()`

Location: `watersupply/calculations/indicators.py:184`

$$\text{CR}\% = \frac{\sum \texttt{collected_meters}}{\sum \texttt{installed_meters}} \times 100 \tag{78}$$

DAG edges: *DAG edge: Metered_Res_Water → CollectionRatio*, *DAG edge: User_Acceptance_WS → CollectionRatio*, *DAG edge: Water_Tariff_Afford → CollectionRatio* — partial (ignores `userAffordability_PCT` and acceptance rate despite the docstring)

5.16 Coverage — `calculate_coverage()`, `CoverageWaterSupply.save()`

Location: `watersupply/calculations/indicators.py:239`; `watersupply/models.py:448`

$$G_{\text{buffer}} = \bigcup_{p \,\in\, \text{pipes}(c)} \text{buffer}(p,\ 500\,\text{m}) \tag{79}$$

$$A_{\text{covered}} = \frac{\text{area}(G_{\text{buffer}})}{10^6}\ [\text{km}^2] \tag{80}$$

$$\text{Coverage}\% = \frac{H_{\text{covered}}}{H_{\text{total}}} \times 100 \tag{81}$$

where H_{covered} counts households in `UsersLocation` points spatially within G_{buffer}, and H_{total} sums `usersTotal` across the city's neighbourhoods. **DAG edges:** *DAG edge: Network → Coverage_WS_Area*, *DAG edge: CityArea → Coverage_WS_Area*, *DAG edge: NumberUsers → Coverage_WS* — implemented

5.17 Non-Revenue Water and ILI

Location: `calculate_nrw()` (`watersupply/calculations/indicators.py:275`); `NonRevenueWater.save()` (`watersupply/models.py:551`)

$$\mathrm{NRW}_{\mathrm{apparent}} = \sum_{\mathrm{type}=A} \texttt{loss_Quantity_m3} \tag{82}$$

$$\mathrm{NRW}_{\mathrm{real}} = \sum_{\mathrm{type}=R} \texttt{loss_Quantity_m3} \tag{83}$$

$$\mathrm{CARL} = \sum_{\mathrm{type}=R,\ \mathrm{year}} \texttt{loss_Quantity_m3} \tag{84}$$

$$\mathrm{UARL} = \sum_{\mathrm{type}=R,\ \mathrm{year}} \texttt{loss_Quantity_m3} \times \frac{\texttt{UnavoidableLossses_PCT}}{100} \tag{85}$$

$$\mathrm{ILI} = \frac{\mathrm{CARL}}{\mathrm{UARL}} \tag{86}$$

The Infrastructure Leakage Index formulation follows the IWA Water Loss Task Force methodology [3]. **DAG edges:** *DAG edge: Real_Losses → NRW, ILI*, *DAG edge: Apparent_Losses → NRW, ILI* — partial (ILI is only defined for Real losses; *DAG edge: Apparent_Losses → ILI* is unimplemented). *DAG edge: Network → Real_Losses* — implemented: every real `NonRevenueWater` record must point (`pipe` FK, enforced in `clean()`) to the `PipeNetwork` where the leakage is presumed, apparent losses may not; `calculate_nrw()` reports `real_losses_by_pipe` (m^3/day and m^3/km/day) and, per admin unit, weights each real loss by the share of its pipe's length inside the unit, splits apparent losses by the unit's share of `UsersLocation.usersTotal`, and recomputes ILI over the unit's real losses.

5.18 Water quality — `calculate_water_quality()`

Location: `watersupply/calculations/indicators.py:144`

$$\mathrm{Compliance}\% = \frac{\sum \texttt{samplesWaterQuality_OK}}{\sum \texttt{samplesWaterQualityTaken}} \times 100 \tag{87}$$

DAG edges: *DAG edge: WT_Efficiency → Samples_WQ*, *DAG edge: Samples_Taken → Samples_WQ*, *DAG edge: Samples_WQ → User_Acceptance_WS* — gap (`acceptance_rate` in the return value is avg(`acceptanceRate`), an independent stored field unrelated to the computed compliance percentage)

5.19 Available fresh water and drought area

Location: `watersupply/calculations/indicators.py:308, 323`

$$\mathrm{AFW} = \sum_{\text{intersecting } u} \texttt{totalQuantity_Mm3} \tag{88}$$

$$A_{\mathrm{drought}} = \sum_{\text{intersecting } u,\ \mathrm{year}} \texttt{areaAffected_km2} \tag{89}$$

DAG edges: *DAG edge: Infiltration → Available_FW*, *DAG edge: Meteorology → Available_FW*, *DAG edge: Total_Extraction → Area_Drought* — gap here: this function only sums the stored `totalQuantity_Mm3`; the stored source records derive their rate, recharge depth and yield (§3.6 and the yearly recharge depth)

5.20 Infiltration indicator (dashboard) — `calculate_infiltration()`, `_infiltration_indicator`

Location: `watersupply/calculations/indicators.py:383`, `watersupply/views.py:177`

The water dashboard's Infiltration card evaluates §3 for the administrative unit u with two what-if controls: the *Rain Event* slider sets $P \in [1, 100]$ mm (default 25; KNMI's daily classes mark *heavy* from 25 mm and *extreme* from 50 mm), and the *Groundwater* switch sets the season s (wet = GHG, default; dry = GLG). P enters only the pure functions and the spatial composition depends on s alone, so the composition could be cached per season; the current implementation recomputes it on every request, together with the rest of `_get_adminUnit_data()`. Displayed: $C_{\text{inf},u}$ (%), $\overline{\text{CN}}$, V_{inf} and V_{run} (m^3), the classified and unclassified areas, the share of the classified area moved to D by shallow groundwater and the share kept as drained, and the texture-only soil-group shares with their rate ranges $[f_{\min}, f_{\max}]$.

DAG edges: *DAG edge: LandCover → Infiltration*, *DAG edge: Soil_Type → Infiltration* — implemented

5.21 Pipe geometry — `PipeNetwork.save()`

Location: `watersupply/models.py:420`

$$L_{\text{km}} = \frac{\text{length}(\texttt{geom})}{1000}, \qquad d_{\text{in}} = \frac{d_{\text{mm}}}{25.4} \tag{90}$$

6 Housing app

Source: `housing/calculations/indicators.py`, `housing/models.py`.

6.1 Supply/demand deficit

Location: `calculate_supply_demand()` (`housing/calculations/indicators.py:14`), `calculate_supply_dema` (`:43`)

$$\text{Deficit} = D - S \tag{91}$$

$$\text{Deficit}\% = \frac{D - S}{D} \times 100 \tag{92}$$

where $S =$ `supply_units` (CBS housing stock, 86098NED) and $D =$ `demand_units` (derived, below). **DAG edges:** *DAG edge: Total_Population → Housing_Demand*, *DAG edge: New_Units → Housing_Supply*, *DAG edge: Housing_Demand → House_Price_Index* — partial (demand is derived, below; `Housing_Demand` → `House_Price_Index` is not implemented; the former `Urbanization` → `Housing_Demand` edge was removed on 2026-11-10)

6.1.1 Household size and housing demand — `HouseholdSize.save()`, `HousingSupplyDemand.save()`

Location: `housing/calculations/chain.py` (the equations, one copy for the stored and the dashboard path), `housing/models.py`, `housing/signals.py`. With P the median population forecast of the city and year (`PopulationProjection`, CBS 85173NED, imported once for the whole project) and H the number of private households (CBS 86291NED, the only housing import), both

whole-gemeente, so the ratio is right even when only some neighborhoods are imported:

$$\bar{s} = P/H \qquad \text{(persons per home, } \texttt{HouseholdSize.average_size}\text{)} \tag{93}$$

$$D_{c,y} = \text{round}\left(P^{\text{base}}_{c,y} / \bar{s}_{c,y}\right) \qquad \text{(}\texttt{demand_units}\text{)} \tag{94}$$

$$\text{residents}(p) = \bar{s}_{c,\text{latest}} \qquad \text{for each dwelling } p \text{ of city } c \text{ (}\texttt{Property.residents}\text{)} \tag{95}$$

$P^{\text{base}}_{c,y}$ is the population stored with that year's `HouseholdSize` (the forecast median, or a manually entered value), else the CBS median forecast (§2), else the imported population. Years before the forecast starts (2023) have no $\bar{s}$ and use the closest year. $\bar{s}_{c,y}$ is the record of year y, else the closest earlier one. A dwelling is a `Property` whose BAG usage includes *Residential function*. Dashboard what-if with population P' and slider value s' (never written; equals D at default controls):

$$D' = D \cdot \frac{P'}{P^{\text{base}}} \cdot \frac{\bar{s}}{s'} \tag{96}$$

Receivers (`housing/signals.py`): `City`/`city_population_changed`, median `PopulationProjection`, `HouseholdSize`, `Property`/`ResidentialProperty`; bulk imports defer them (`deferred_housing_refresh`). **DAG edges:** *DAG edge: Total_Population → Housing_Demand*, *DAG edge: Household_Size → Housing_Demand*, *DAG edge: Household_Size → Property* — implemented

6.2 Pipeline units — `calculate_new_units()`

Location: `housing/calculations/indicators.py:66`

$$U_{\text{completed}} = \sum_{\texttt{year_expected_completion} \le y} \texttt{project_units} \tag{97}$$

$$U_{\text{pipeline}} = \sum_{\texttt{year_expected_completion} > y} \texttt{project_units} \tag{98}$$

DAG edges: *DAG edge: Zoning_Status → New_Units*, *DAG edge: Network → New_Units*, *DAG edge: New_Units → Housing_Supply* — partial (counts completions/pipeline; never reads `Zoning_Status`/`Network` as inputs). The land-cover side is `calculate_pipeline_land_take()` below.

6.2.1 Pipeline land take — `calculate_pipeline_land_take()`

Location: `housing/calculations/indicators.py`. With $F_c = \bigcup$ footprints of the pipeline projects (`year_expected_completion` $> y$) of city c, overlapping projects counted once, and $\mathcal{L}_c$ the city's latest land-cover year:

$$A_k = \sum_{c} \sum_{P \in \mathcal{L}_c,\ \text{class}(P)=k} |P \cap F_c| \tag{99}$$

$$A_{\text{greenfield}} = \sum_{k \notin \mathcal{U}} A_k, \qquad A_{\text{infill}} = \sum_{k \in \mathcal{U}} A_k, \qquad A_{\text{uncovered}} = \textstyle\sum_c |F_c| - \sum_k A_k \tag{100}$$

where $\mathcal{U}$ are the classes that already count as urban area (*urban fabric, road, rail, transport*). Greenfield is the land the new units would turn into urban fabric. Dashboard only (housing *New Housing Units* card), nothing is written: observed land cover is not overwritten by plans. **DAG edge:** *DAG edge: New_Units → LandCover* — implemented

6.3 Mortgage indicators — `calculate_mortgage_indicators()`

Location: `housing/calculations/indicators.py:123`

$$\overline{m} = \text{avg}(\texttt{monthlyPayment}), \quad \bar{\iota} = \text{avg}(\texttt{interestRate}), \quad \overline{L} = \text{avg}(\texttt{totalLoanAmount}) \tag{101}$$

plus province-level policy fields (`interest_rate`, `LTV_limit`, `LTI_limit`, `mortgageRate`) read as-is. **DAG edges:** *DAG edge: Central_Bank → Mortgage*, *DAG edge: Credit_Supply → Mortgage*, *DAG edge: Mortgage → Affordability_Stress*, *DAG edge: Mortgage → Income_Expenses* — gap for the two outgoing edges (averages are exposed on the dashboard but never fed back into `HousingAffordability`)

6.4 House Price Index — `calculate_house_price_index()`

Location: `housing/calculations/indicators.py:200`

$$\Delta_{\text{YoY}}\% = \frac{\overline{\text{HPI}}_y - \overline{\text{HPI}}_{y-1}}{\overline{\text{HPI}}_{y-1}} \times 100 \tag{102}$$

DAG edges: *DAG edge: Housing_Demand → House_Price_Index*, *DAG edge: Property → House_Price_Index* — gap (`HousePriceIndex` has no `save()`; `index_value` is a flat stored field, and this function only averages it)

6.5 Affordability index — `HousingAffordability.save()`

Location: `housing/models.py:150`

$$\text{AI} = \frac{\texttt{medianHousePrice}}{\texttt{medianIncome}} \tag{103}$$

$$\text{Income}_{\text{disposable}} = \texttt{medianIncome} - \texttt{medianExpenditure} \tag{104}$$

The house-price-to-income ratio formulation is the standard affordability metric used in comparative housing-affordability studies (e.g. Demographia's Median Multiple) [4]. **DAG edges:** *DAG edge: Property → Affordability_Stress*, *DAG edge: Income_Expenses → Affordability_Stress*, *DAG edge: Mortgage → Affordability_Stress*, *DAG edge: Water_Tariff_Afford → Income_Expenses*, *DAG edge: Rent → Income_Expenses*, *DAG edge: Mortgage → Income_Expenses* — gap (computed only from this model's own stored fields; never reads `Mortgage`, `Rentals`, or watersupply tariff data — an open TODO in `housing/models.py` also questions whether disposable income should replace median income in the numerator's denominator)

6.6 Zoning composition — `calculate_zoning()`

Location: `housing/calculations/indicators.py:297`; groups `ZONING_GROUPS` (`:289`)

$$\text{Area}\%_t = \frac{\sum_{z\,:\,\texttt{zone_type}(z) \in K_t} \texttt{area}(z)}{\sum_{\text{all zones}} \texttt{area}(z)} \times 100 \tag{105}$$

`ZoningArea.zone_type` is a foreign key to the HILUCS codelist (§3), filled from each INSPIRE zoning element's `hilucsLandUse` reference. The four dashboard groups are code sets with all their sub-codes (`hilucs_q()`): $K_{\text{residential}} = \text{sel}(5) \setminus \text{sel}(5.2)$, $K_{\text{commercial}} = \text{sel}(3.1) \cup \text{sel}(3.2)$, $K_{\text{industrial}} = \text{sel}(2)$, $K_{\text{mixed}} = \text{sel}(5.2)$. Other land uses (e.g. water, agriculture) count in the denominator only. **DAG edges:** *DAG edge: Housing_Demand → Zoning_Status*, *DAG edge: Zoning_Status → Property*, *DAG edge: Zoning_Status → New_Units* — gap (descriptive aggregation only; no model consumes `Zoning_Status` as an input)

6.7 Property market and vacancy — `calculate_property_indicators()`

Location: `housing/calculations/indicators.py:236`

$$\overline{p} = \text{avg}(\texttt{salePrice_EUR}), \qquad \overline{p}_{\text{EUR/m}^2} = \text{avg}(\texttt{unitaryPrice_EUR_per_sqm}), \qquad \overline{v} = \text{avg}(\texttt{vacancyRate}) \tag{106}$$

DAG edges: *DAG edge: Housing_Supply → Property*, *DAG edge: Zoning_Status → Property*, *DAG edge: Accessibility → Property*, *DAG edge: Property → Affordability_Stress*, *DAG edge: Property → Rent*, *DAG edge: Property → House_Price_Index*, *DAG edge: Property → Mortgage* — gap (no `Accessibility` model exists; none of the outgoing edges are computed — this function only averages stored fields)

7 Urban heat app

Source: `urban_heat/calculations.py`. Thermal-comfort rasters (SVF, T_{mrt}, PET, UTCI) are computed on demand for the map view with SOLWEIG [5] (Section 7.6) or imported pre-computed; the dashboard statistics below only read and classify them. LST is computed from Landsat and CrossTwin's NDVI layer (Section 7.5).

7.1 Green area and vegetation coverage

Location: `calculate_green_area()` (`urban_heat/calculations.py:23`), `calculate_vegetation_coverage()` (`:79`)

$$A_{\text{parks}} = \Big| \bigcup_{p \in \text{parks}} p \cap U \Big| \tag{107}$$

$$A_{\text{green,total}} = \Big| \Big(\bigcup_{p \in \text{parks}} p \cup \bigcup_{l \in \text{green classes},\ y_l = y_{\max}(\text{city}_l)} l \Big) \cap U \Big| \tag{108}$$

$$A_{\text{green,lc}} = A_{\text{green,total}} - A_{\text{parks}} \tag{109}$$

$$\text{VegCov\%} = \frac{A_{\text{green,total}}}{A_{\text{unit}}} \times 100 \tag{110}$$

with parks the park space: `nature.GreenSpaces` rows of type `park` plus the BGT recreation areas (type `recreatie...`, `GreenSpaces.objects.park_space()`), U the unit's geometry and $|\cdot|$ the area in m^2 (a polygon lying entirely within U contributes $|l|$ without being clipped). Green classes are those whose name contains one of `GREEN_LANDCOVER_KEYWORDS`; only each city's most recent land-cover year $y_{\max}$ counts. The stored `percentage` (a share of the province) is no longer used, so the result is valid for any unit level. Parks and green land cover are merged (`ST_UnaryUnion`) after clipping and before measuring, so ground under both, or under two overlapping parks, counts once; $A_{\text{green,lc}}$ is the green land cover outside the parks, and $A_{\text{parks}} + A_{\text{green,lc}} = A_{\text{green,total}}$.**DAG edges:** *DAG edge: LandCover → Green_Area*, *DAG edge: Green_Area → Vegetation_Coverage* — partial; *DAG edge: Vegetation_Coverage → DSM* — implemented (Section 2.13)

7.2 Street canyon aspect ratio — `builtup.StreetCanyon`, `calculate_canyon_aspect()`

Location: `builtup/calculations/chain.py` (equations), `builtup/calculations/canyon.py::street_inputs` (inputs, one SQL statement for many streets), `builtup/signals.py` (receivers), `urban_heat/calculations.py::c` (dashboard). Each street line is trimmed by 10 m at both ends of every part (the buildings of a

crossing street do not count) and cut into pieces s of at most 100 m. Each piece gets a single-sided buffer L_s, R_s of 30 m on its left and right. With b the buildings, h_b = `height_m`, else 3 m× `numberFloors` (buildings with neither are skipped):

$$H_s^{\text{side}} = \frac{\sum_b |b \cap \text{side}_s|\, h_b}{\sum_b |b \cap \text{side}_s|} \quad \text{(0 without buildings)}, \quad d_s = \text{dist}(s, \text{nearest } b \text{ left}) + \text{dist}(s, \text{nearest } b \text{ right}) \tag{111}$$

and per street, with ℓ_s the piece lengths,

$$H = \frac{1}{2}\left(\frac{\sum_s \ell_s H_s^L}{\sum_s \ell_s} + \frac{\sum_s \ell_s H_s^R}{\sum_s \ell_s}\right), \quad W = \begin{cases} \texttt{Street.width} & \text{if set} \\ \frac{\sum_{s:d_s} \ell_s d_s}{\sum_{s:d_s} \ell_s} & \text{if some piece has buildings on both sides} \\ 8/12/20\,\text{m} & \text{residential / secondary / primary} \end{cases}, \quad \frac{H}{W} \tag{112}$$

An open side counts as height 0 (Oke's symmetric canyon [21]). The result is stored per street (`StreetCanyon`, a map layer along the street line). A saved or deleted street refreshes its own canyon; a building created, moved, reshaped, re-heighted or deleted refreshes the canyons of the streets within 30 m of its old and new footprint (one set-based upsert, deferred to the end of an import, replayed by the bulk `Street` finalizer). Editor changes are journalled, so Restore reverts them. The dashboard's unit value weights each canyon by the length of its street inside the unit U:

$$\left.\frac{H}{W}\right|_U = \frac{\sum_i (H/W)_i\, |l_i \cap U|}{\sum_i |l_i \cap U|} \tag{113}$$

with the shares of that length below 0.5 (open) and at or above 2 (deep). Without stored canyons it falls back to $\overline{h}_{\text{buildings}}/\overline{\texttt{width}}_{\text{streets}}$. **DAG edges:** *DAG edge: Buildings → Canyon_Aspect*, *DAG edge: Streets → Canyon_Aspect* — implemented. *DAG edge: Canyon_Aspect → DSM* was removed from the DAG: H/W is read off the buildings, which reach the DSM through *DAG edge: Buildings → DSM*.

7.3 Raster statistics — `_raster_stats_sql()`, `get_thermal_indices()`

Location: `urban_heat/calculations.py:112, 145`

$$\{\min, \max, \mu, \sigma\} = \text{ST_SummaryStats}\left(\text{ST_Clip}(R,\ \texttt{geom}(u))\right) \tag{114}$$

computed via PostGIS raster functions over the most recently dated raster of each type (LST, T_{mrt}, PET, UTCI, WBGT, SVF, SUHII) intersecting the admin unit. **DAG edges:** *DAG edge: Meteorology → Tmrt, PET, UTCI*, *DAG edge: DSM → SVF → Tmrt, PET*, *DAG edge: LST → PET*, *DAG edge: Tmrt → PET, UTCI* — gap (statistics *over* existing rasters only; no raster is derived from another)

7.4 Thermal stress classification

Location: `classify_pet()` (:201), `classify_utci()` (:225), `classify_wbgt()` (:267)

PET thresholds (in °C) follow the physiologically-based 9-class scheme of Matzarakis and Mayer [8]:

$$\text{class(PET)} = \begin{cases} \text{Very Cold Stress} & \text{PET} < 4 \\ \text{Cold Stress} & 4 \le \text{PET} < 8 \\ \text{Slight Cold Stress} & 8 \le \text{PET} < 13 \\ \text{No Thermal Stress (cool)} & 13 \le \text{PET} < 18 \\ \text{Comfortable} & 18 \le \text{PET} < 23 \\ \text{No Thermal Stress (warm)} & 23 \le \text{PET} < 29 \\ \text{Slight Heat Stress} & 29 \le \text{PET} < 35 \\ \text{Moderate Heat Stress} & 35 \le \text{PET} < 41 \\ \text{Extreme Heat Stress} & \text{PET} \ge 41 \end{cases} \tag{115}$$

UTCI thresholds follow the 10-class scheme defined by the COST Action 730 UTCI framework [15]:

$$\text{class(UTCI)} = \begin{cases} \text{Extreme Cold Stress} & \text{UTCI} < -40 \\ \vdots & \vdots \\ \text{Extreme Heat Stress} & \text{UTCI} \ge 46 \end{cases} \quad \text{(9 intermediate bands; see } \texttt{urban_heat/calculations.py} \tag{116}$$

WBGT bands (°C: 25/28/30/33 thresholds) are marked **provisional** in the source (`urban_heat/calculations.py` pending confirmation against a named standard.

7.5 Land surface temperature, Statistical Mono-Window — `urban_heat/lst/`

Location: `urban_heat/lst/smw.py` (equations), `gee.py` (inputs), `pipeline.py::calculate_lst()`; run from the map's LST tool (Actions menu, over the area in view: the view clipped to the NDVI layers' coverage, overlapping NDVI layers merged newest first, scenes ranked by the clear share of the view) or with `manage.py calculate_lst`. Recalculating a scene replaces its earlier rows inside the new extent.

Python port of the Earth Engine modules of Ermida et al. [6] (`users/sofiaermida/landsat_smw_lst`), with the Landsat NDVI replaced by CrossTwin's NDVI layer (`physicalEnv.LandCoverRaster`, `index = NDVI`, Sentinel-2, averaged onto a 30 m grid in EPSG:28992 snapped to the layer's extent). Earth Engine supplies the Landsat 9 TIRS band-10 TOA brightness temperature T_b (K), the Level-2 `QA_PIXEL`, ASTER GED emissivities $\varepsilon_{13}, \varepsilon_{14}$ and NDVI, and NCEP precipitable water.

$$\text{FVC} = \text{clip}_{[0,1]}\left(\left(\frac{\text{NDVI} - 0.2}{0.86 - 0.2}\right)^2\right) \tag{117}$$

$$\varepsilon_{k,\text{bare}} = \frac{\varepsilon_k - 0.99\,\text{FVC}_{\text{ASTER}}}{1 - \text{FVC}_{\text{ASTER}}}, \qquad k \in \{13, 14\} \tag{118}$$

$$\varepsilon = 0.99\,\text{FVC} + (1 - \text{FVC})\left(0.6820\,\varepsilon_{13,\text{bare}} + 0.2578\,\varepsilon_{14,\text{bare}} + 0.0584\right) \tag{119}$$

$$\text{TPW} = \text{TPW}_1\,\frac{\Delta t_2}{6\,\text{h}} + \text{TPW}_2\,\frac{\Delta t_1}{6\,\text{h}}, \qquad i = \min\left(9, \max(0, \lceil \text{TPW}/6 \rceil - 1)\right) \tag{120}$$

$$\text{LST} = A_i \frac{T_b}{\varepsilon} + \frac{B_i}{\varepsilon} + C_i \tag{121}$$

$\text{TPW}_{1,2}$ are the two water-vapour fields of the acquisition day closest in time ($\Delta t_{1,2}$ their distances, in units of the source's field spacing: 6 h for NCEP, 1 h for ERA5). NCEP reanalysis (`NCEP_RE/surface_wv`), which the original code uses, ended on 2026-03-18; later scenes use

ERA5 hourly `total_column_water_vapour`, which over Enschede differs from NCEP by about 1–4 mm, mostly within one 6 mm coefficient bin. The source used is stored in `measurement_method`. (A_i, B_i, C_i) is the Landsat 9 row of the SMW table for TPW bin i (`coefficients.py`). Water pixels (QA bit 7) get $\varepsilon = 0.99$, snow/ice (bit 5) 0.989; cloud and cloud shadow (bits 3, 4), TPW < 0 (masked water-vapour cell) and cells outside the NDVI layer are nodata. The clip in FVC is applied after squaring, as in the published code, so NDVI < 0.2 gives a small positive FVC. With `--aster-emissivity` the vegetation correction is skipped: $\varepsilon = 0.6820\,\varepsilon_{13} + 0.2578\,\varepsilon_{14} + 0.0584$. Output: one `LandSurfaceTemperature` raster in °C, nodata −9999, COG via `core/signals.py`. Computed on demand, not by a receiver (it downloads from Earth Engine). **DAG edges:** *DAG edge: RS_Imagery → LST*, *DAG edge: LandCover → LST* — implemented

7.6 Thermal comfort with SOLWEIG — `urban_heat/solweig/`

Location: `urban_heat/solweig/inputs.py` (input rasters), `met.py` (meteorology), `pipeline.py` (run, PET, storage); run from the map's thermal comfort tool (Actions menu, over the area in view and one hour) as a background `ThermalComfortRun`. Full description and process graph: `docs/THERMAL_COMFORT.md`.

Inputs on one grid. The view is extended by a halo of 50 m (shadows of buildings outside the view), read from the DSM with the editor's building patches (Section 2.12) at a cell size Δ (default 1 m) and split into SOLWEIG's rasters:

$$\text{Building_DSM}(x) = \begin{cases} \max(z(x), g(x)) & x \in \bigcup \text{building footprints} \\ g(x) & \text{otherwise} \end{cases} \tag{122}$$

$$\text{Trees}(x) = \begin{cases} z(x) - g(x) & x \notin \text{footprints},\ z(x) - g(x) > 2.5\,\text{m} \\ 0 & \text{otherwise} \end{cases} \tag{123}$$

with g the ground as above. The land cover of the latest year is mapped to SOLWEIG's classes (1 paved, 2 buildings, 5 grass, 6 bare soil, 7 water; class-name keywords at word starts, anything else paved), buildings burned last. Windows above 10^6 cells are refused.

Meteorology. One hourly record (air temperature T_a, relative humidity RH, wind speed v and direction, global radiation $K_\downarrow$, pressure p), uniform over the view: entered by hand, or from the KNMI station nearest to the view centre that measured that hour (validated hourly data; T and v in 0.1 units, $K_\downarrow = Q \cdot 10^4/3600$ with Q in J/cm^2 per hour, p in 0.1 hPa), or from ERA5 hourly at the view centre [12]:

$$\text{RH} = 100\,\frac{e_s(T_d)}{e_s(T_a)}, \qquad e_s(T) = 6.1094\exp\!\left(\frac{17.625\,T}{T + 243.04}\right) \tag{124}$$

$$v = \sqrt{u^2 + v_n^2}, \qquad \theta = \operatorname{atan2}(-u, -v_n) \bmod 360^\circ \tag{125}$$

$$K_\downarrow = \text{ssrd}/3600\,\text{s} \tag{126}$$

(Magnus form of [13]; θ is the direction the wind blows from). The record is written as a UMEP met file for the hour before and the analysed hour, in local time at the view centre.

SOLWEIG. `solweig-gpu` [14] (PyTorch; CUDA when available) computes wall heights and aspects, the sky view factors and shadow matrices (cached per input hash), and SOLWEIG 2022a's radiant fluxes, mean radiant temperature T_{mrt} and UTCI [5, 15]. The solar position comes from the centre of the input grid and its timezone on that date (SPA), so it follows the view. The analysed hour is band 2; the halo is cropped off.

PET. `pipeline.urban_pet` calls the `pet-adjusted` package (`urban_pet_from_rasters`) on the SOLWEIG window, before the halo is cropped. It solves the steady-state model of [10] as corrected by [11] (a vectorised port of `pythermalcomfort.pet_steady`; standard person: 35 years, 1.75 m, 75 kg, 0.9 clo, 1.37 met) per cell, with air temperature and wind adjusted to the street after [9]:

$$T_a(x) = T_{ref} + \mathrm{UHI}_{max}(x)\, f(d,h), \qquad \mathrm{UHI}_{max}(x) = \left(2 - \overline{\mathrm{SVF}}(x) - \overline{F_{veg}}(x)\right) \left(\frac{S_{\downarrow}}{\rho c_p} \frac{(T_{max} - T_{min})^3}{\bar{u}} \right)^{1/4} \tag{127}$$

$$v(x) = u_{10} \left((u_{1.2}/u_{10})(\lambda_b, \lambda_t, H) - 0.0796 \right) 0.9175 + 0.1254\, u_{10} \tag{128}$$

$$\mathrm{PET}(x) = \mathrm{PET}\big(T_a(x), T_{mrt}(x), v(x), \mathrm{RH}, p\big) \tag{129}$$

$\overline{\mathrm{SVF}}$ and $\overline{F_{veg}}$ are averaged over the cell's upwind source area, $f(d,h)$ is the diurnal factor of the date and UTC hour (1 April – 30 September only), $S_{\downarrow}$, T_{max}, T_{min} and $\bar{u}$ are the day's mean shortwave radiation, extreme temperatures and mean wind (`met.Met` day fields, from the KNMI station or ERA5), and λ_b, λ_t, H are the frontal area densities of buildings and trees and their mean height (Macdonald). DSM = `Building_DSM` + `Trees`; vegetation = SOLWEIG's trees and grass. Without the day's values or outside the diurnal table's months, $T_a = T_{ref}$ (the 1.2 m wind is kept). Building cells are nodata.

Output: one `MeanRadiantTemperature`, `UTCI` and `PET` raster in °C (nodata −9999, COGs via `core/signals.py`), replacing earlier SOLWEIG rows of the same hour inside the view. Computed on demand, not by a receiver. **DAG edges:** *DAG edge: DSM → SVF → Tmrt, PET*, *DAG edge: Meteorology → Tmrt, PET, UTCI*, *DAG edge: Tmrt → PET, UTCI* — implemented (on demand; meteorology uniform over the view for Tmrt and UTCI; PET has the urban heat island and 1.2 m wind, with source areas truncated at the window's 50 m halo)

7.7 Nature-Based Solutions coverage — `calculate_nbs_coverage()`

Location: `urban_heat/calculations.py:283`

$$A_{\mathrm{NBS}} = \sum_{\text{NBS polygons} \cap \texttt{geom}(u)} \texttt{area} \tag{130}$$

DAG edges: *DAG edge: UTCI → NBS*, *DAG edge: PET → NBS* — gap (counts NBS geometries only; never reads PET or UTCI values to relate NBS placement to thermal stress)

8 Map editing and what-if backup (mainMap app)

Source: `core/editing.py`, `core/editable_layers.toml`, `mainMap/models.py`, `core/static/js/Editing.js`; design in `docs/MAP_EDITING.md`. Staff users edit built-up objects (`Building`, `Street`, `Facility`, `ZoningArea`, and green spaces/parks as `nature.GreenSpaces`) directly on the map, and can set one restore point so what-if edits can be undone. These functions implement no DPSIR edge themselves; they are *Response*-side tooling that changes the State variables (buildings, streets, zoning) from which the indicator functions above are computed. Every write goes through the model's `save()`, so the derivations documented elsewhere in this registry (e.g. `Building.save()`, the cache invalidation in `core/signals.py`) apply unchanged.

8.1 Editable-layer allowlist — `EDITABLE_LAYERS`, `is_editable()`

Location: `core/editable_layers.toml`, `core/editing.py:42`, `:79`

For a layer ℓ and user u,

$$\text{editable}(\ell, u) = [\, \ell \in \mathcal{E} \,] \wedge [\, u.\texttt{is_staff} \,], \tag{131}$$

where $\mathcal{E}$ is the allowlist. Each entry lists the fields the model derives itself (`exclude`, kept out of the form) and the fields the form must not require (`optional`). For `Building`: `exclude` = {`area_sqm`, `neighborhood`, `last_updated`, `connectivity`}, `optional` = {`buildingType`}; for `ZoningArea`: `exclude` = {`area`}. The layer catalog (`available_layers`) adds `editable` and `edit_geom_type` to an entry only when editable(ℓ, u) holds.

8.2 Attribute and geometry form — `_build_form()`, `feature_form()`

Location: `core/editing.py:95`, `:158`

The browser sends the edited shape as WGS84 GeoJSON in a hidden form field. Django's GIS form field parses it, rejects a geometry whose type differs from the model field's (e.g. a `Polygon` for a `MultiPolygonField`), and reprojects it to the storage CRS:

$$g_{\text{stored}} = T_{4326 \to 28992}(g_{\text{GeoJSON}}) \tag{132}$$

The editor (`Editing.js::_writeGeometry()`) wraps single geometries before sending them when the field is a Multi* type: `Polygon`$(c) \mapsto$ `MultiPolygon`$([c])$. A form posted without a new shape sends back the stored EWKT, which already carries SRID = 28992, so no transformation (and no coordinate drift) occurs. For display, the stored geometry is sent at full resolution as $T_{28992 \to 4326}(g_{\text{stored}})$, not the map's copy, which is simplified below zoom 14 (`_geojson_sql()`).

Saving calls `obj.save()`, so for a `Building` the existing derivations run: `area_sqm` = area(g_{stored}) in m^2, `neighborhood` by point-in-polygon of the centroid, and `buildingType` from `usageFunction` (`builtup/models.py`).

8.3 Identifiers for new objects — `_next_negative_pk()`

Location: `core/editing.py:108`

`Building.id` is the BAG identifier, not generated by the database. A user-created building gets

$$\text{id}_{\text{new}} = \min\left(0, \min_{b \in \texttt{Building}} \text{id}_b\right) - 1, \tag{133}$$

so new objects count down from -1, never collide with (positive) BAG ids, and are recognisable as user-made. Models with a database-generated key (`AutoField`) keep it.

8.4 Deferred-constraint check — `_check_constraints()`

Location: `core/editing.py:118`

Django creates foreign keys as `DEFERRABLE INITIALLY DEFERRED`, and the project's spatial FKs use `on_delete=DO_NOTHING`, so a deletion that leaves a dangling reference (a `Property` pointing at a deleted `Building`) would only fail at `COMMIT`. `SET CONSTRAINTS ALL IMMEDIATE` checks all pending references inside the atomic block; a violation raises `IntegrityError`, rolls back just that operation and is reported as a message. Used by `feature_delete()` (`:176`) and `restore_backup()`.

8.5 Change journal — `journal()`

Location: `core/editing.py:240`; models `EditBackup`, `EditBackupRow` (`mainMap/models.py`)

A backup B is created empty at time t_0; nothing is copied. Let $s_t(o)$ be the stored row of object o at time t, with $s_t(o) = \varnothing$ if o does not exist. Before the first editor change to o after t_0, the journal records

$$J_B(o) = \begin{cases} s_{t_0}(o) & \text{if } o \text{ is edited or deleted} \\ \varnothing & \text{if } o \text{ is created} \end{cases} \qquad \text{only if } o \notin \mathrm{dom}(J_B), \tag{134}$$

i.e. the first pre-image wins: an object edited n times is still restored to its state at t_0. Rows are serialized with Django's serializer, which keeps the geometry in its stored CRS. The bar in the Layers panel reports $|\mathrm{dom}(J_B)|$ as "objects changed". Without an active backup, J is not written and edits are permanent. At most one backup exists at a time.

8.6 Restore — `restore_backup()`

Location: `core/editing.py:293`

In one transaction (wrapped in `importer/batching.py::deferred_cascades()`), for every $o \in \mathrm{dom}(J_B)$:

$$s_{\text{after}}(o) = J_B(o) = \begin{cases} s_{t_0}(o) & \text{(raw save: update, or re-insert with the original id)} \\ \varnothing & \text{(delete the object created after } t_0\text{)} \end{cases} \tag{135}$$

Re-insertions and updates run before deletions so that reference targets exist again first. The raw save writes the stored values back exactly, derived fields included, without re-running `save()`, and still fires `post_save`, which bumps the layer's GeoJSON cache version. If `_check_constraints()` fails, the whole restore is rolled back (never partial); otherwise B and J_B are deleted and the affected layer keys are returned for the map to reload. Objects changed only outside the editor (importer, admin, QGIS, psql) are not in $\mathrm{dom}(J_B)$ and are left as they are.

Caveat: receivers that skip raw saves (`watersupply/signals.py`: `if raw: return`) would not fire on restore. Built-up models have none; adding such a layer to $\mathcal{E}$ requires triggering its cascades explicitly.

Side effects of an edit. Rows that another app changes because of an edit are journalled too, through custom signals that `mainMap/signals.py` subscribes to. Creating a building runs `urbanize_around()` (§2.9), which sends `landcover_rewriting` with the land-cover rows $\mathcal{A}$ it is about to clip or delete and `landcover_created` with the new urban-fabric row u; the receivers add $J_B(p) = s_{t_0}(p)$ for $p \in \mathcal{A}$ and $J_B(u) = \varnothing$ under the key of the signal's sender (`physicalEnv.LandCoverVector`, or `physicalEnv.UrbanFabricCut` for the stored cuts). Moving or deleting a drawn building runs `release_ring()`, which journals the same way. Restore therefore re-inserts the clipped and deleted polygons and cuts with their original geometry and deletes u, and the land-cover receiver recomputes the neighbourhoods' urban area inside the same deferred block. Only keys in the vector registry are returned as layers to reload.

8.7 Map-side editor — `Editing.js`

Location: `core/static/js/Editing.js`; hooks in `Layers.js` (`reloadLayer()`, `_popupEditButton()`), `MapUI.js::renderLayerList()`, `mainMap.js::refreshActivePanel()`, `Map_init.js`

No equations; behaviour relevant to reproducing results:

- `startEdit()` / `startCreate()` load the form into the side panel; a `MutationObserver` on `#panel-body` attaches Mapbox GL Draw when the form appears and detaches it when the panel content is replaced.
- While editing, the original object is hidden with the filter `id` $\neq$ pk, feature popups, admin-unit clicks and panel refreshes are suspended, and the camera pitch is set to 0.
- The `feature-saved` and `backup-restored` HTMX events reload the affected layers (`reloadLayer()`), so the map always shows the stored state.

9 Imported climate-effect indicators (importer app, Klimaateffectatlas)

Source: `importer/external_catalog.py` (`kea_*` entries, `_kea_neighborhood()`, `_kea_building()`), `importer/external_data.py` (`_prepare_features()`, `_keep_existing()`, `_enrich_rows()`); target models `builtup.NeighborhoodClimateRisk`, `urban_heat.NeighborhoodHeat`, `housing.NeighborhoodSocialPro` and new `builtup.Building` fields. The layer-by-layer mapping, and the 47 of 68 WFS layers that are not imported, are listed in `docs/KEA_WFS_MAPPING.md`. No equation of CrossTwin produces these values: they are stored inputs, transformed only on import.

9.1 Value transforms — `_prepare_features()`

For a source attribute x of feature f:

$$x' = \begin{cases} \varnothing & x \in \mathcal{N} \\ m(x) & x \in \mathrm{dom}(m) \\ k\,x & \text{for a scaled attribute} \\ x & \text{otherwise} \end{cases} \tag{136}$$

with $\mathcal{N} = \{-99995, \ldots, -99999\}$, the RIVM health monitor's no-value codes (frail health, loneliness, rental share, socio-economic status); $k = 100$ for the pile-rot risks, which KEA delivers as fractions (0–0.054 in Enschede), while settlement risk is already a percentage (0.9–17.7) and is stored as delivered; and m the drought-sensitivity legend, $m(\mathrm{W}) = 2$ (weinig gevoelig, Low), $m(\mathrm{G}) = 3$ (gevoelig, Medium), $m(\mathrm{Z}) = 4$ (zeer gevoelig, High), with a missing value left to the model default 0 (not groundwater-dependent). A building-block feature whose `BAG.Id` lists n panden is split into n features, each id left-padded to the 16 digits of a BAG identifier, so that every pand of the block receives the block's facade-shade score.

9.2 Join and write rules — `_keep_existing()`, `_enrich_rows()`

Let K be the set of stored keys of the join model (`Neighborhood.id` or `Building.id`) and $k(f)$ the feature's key (`buurtcode`, `identificatie`, ...). Only features with $k(f) \in K$ are written; the others are counted in the import message. Neighbourhood layers upsert one row per neighbourhood on `neighborhood_id`, each layer writing only its own columns. Building layers update the existing row only, $s'(b) = s(b) \oplus \{\text{mapped attributes of } f\}$, never insert and never write the geometry; they use `update()` and bump the map cache of every registry model on the building table (including the `Energy.BuildingEnergyLabel` proxy).

DAG edges: none computed. The values inform `PET`, `Vegetation_Coverage` (heat and canopy), and the context of `Income_Expenses`/`Affordability_Stress` (social profile) — stored inputs.

10 Summary table

App	Function	DAG edge(s) (abbrev.)	Status
administrative	`extrapolate()`	Population_Growth → Total_Pop	implemented
administrative	`_city_growth_factor()`	Population_Growth → Total_Pop	implemented
administrative	`annual_growth_rate()`	Population_Growth → Total_Pop	implemented
administrative	`City.save()` / `Province.save()`	(population cascade)	implemented
physicalEnv	`urbanize_around()` / `release_ring()`	New_Units → LandCover (editor buildings)	implemented
physicalEnv	`dsm_patches.record_change()` / `apply_patches()`	Buildings → DSM (editor buildings)	implemented
physicalEnv	`dsm_patches.record_vegetation_change()`	Vegetation_Coverage → DSM (editor trees, forests)	implemented
builtup	`StreetCanyon.save()` / `signals.refresh_canyons()`	Buildings / Streets → Canyon_Aspect	implemented
builtup	`degurba.classify()` / `Urbanisation.save()`	Total_Population / Buildings / LandCover → Degree_Urbanisation	implemented
physicalEnv	`urbanize_around()`	Buildings → LandCover	implemented
physicalEnv	`_recompute_neighborhood_urban_area()`	LandCover → CityArea	implemented
—	—	New_Units → Buildings, Buildings → Streets, Streets → LandCover	gap
watersupply	`calculate_total_demand()` / `TotalWaterDemand.save()`	Total_Pop → Total_Water_Demand	implemented
watersupply	`ConsumptionCapita.save()`	Total_Pop → Total_Consumption	implemented
watersupply	`calculate_total_extraction()`	Available_FW → Total_Extraction	implemented
watersupply	`calculate_total_production_day()`	Total_Extraction / Imported_Water → Total_Water_Prod	implemented
watersupply	`calculate_supply_security()` / `SupplySecurity.save()`	Total_Water_Demand / Prod → Supply_Security → Service_Time	implemented
watersupply	`calculate_energy_consumption()`	Total_Extraction / Prod → Energy_Consumption	partial
watersupply	`ExtractionWater.save()` (emissions)	Total_Extraction → CO2_Emission	implemented
watersupply	`OPEX.save()`	Extraction / Imported / WT_Cost / Network → OPEX	implemented
watersupply	`calculate_opex_recovery()`	OPEX / CollectionRatio / NRW → OPEX_Recovery	partial
watersupply	`MeteredResidential.save()`	Metered_Res_Water → OPEX_Recovery	implemented
watersupply	`calculate_collection_ratio()`	Metered_Res_Water / User_Acceptance / Water_Tariff → CollectionRatio	partial
watersupply	`calculate_coverage()` / `CoverageWaterSupply.save()`	Network / CityArea → Coverage_WS_Area	implemented
watersupply	`calculate_nrw()` / `NonRevenueWater.save()`	Real / Apparent_Losses → NRW, ILI	partial
watersupply	`NonRevenueWater.pipe` / `calculate_nrw()`	Network → Real_Losses	implemented
watersupply	`calculate_water_quality()`	Samples_WQ → User_Acceptance_WS	gap
watersupply	`calculate_available_freshwater()`	Infiltration / Meteorology → Available_FW	gap

App	Function	DAG edge(s) (abbrev.)	Status
physicalEnv	`weighted_infiltration_rate_mm_h()` / `AvailableFreshWater.save()`	LandCover / Soil_Type → Infiltration (stored)	implemented
watersupply	`recharge_depth_cm_yr()` / `AvailableFreshWater.save()`	Infiltration / Meteorology → Available_FW	implemented
physicalEnv	`classify_soil_group()` / `SoilType.save()`	Soil_Type → Infiltration	implemented
physicalEnv	`soil_landcover_composition()`	LandCover / Soil_Type (+ groundwater) → Infiltration	implemented
physicalEnv	`scs_runoff_mm()` / `summarize_infiltration()`	LandCover / Soil_Type → Infiltration	implemented
watersupply	`calculate_infiltration()` (dashboard)	LandCover / Soil_Type → Infiltration	implemented
housing	`calculate_supply_demand()`	Total_Population → Housing_Demand	implemented
housing	`calculate_new_units()`	New_Units → Housing_Supply	partial
housing	`calculate_pipeline_land_take()` (dashboard)	New_Units → Buildings → LandCover	implemented
housing	`calculate_mortgage_indicators()`	Central_Bank / Credit_Supply → Mortgage	partial
housing	`calculate_house_price_index()`	Housing_Demand / Property → HPI	gap
housing	`HousingAffordability.save()`	Rent / Mortgage / Water_Tariff → Income_Expenses → Affordability_Stress	gap
housing	`calculate_zoning()`	Zoning_Status → Property / New_Units	gap
housing	`calculate_property_indicators()`	Housing_Supply / Accessibility → Property	gap
urban_heat	`calculate_green_area()` / `calculate_vegetation_coverage()`	LandCover → Green_Area → Vegetation_Coverage	partial
urban_heat	`calculate_urban_morphology()`	reads stored Canyon_Aspect	implemented
urban_heat	`get_thermal_indices()`	DSM → SVF → Tmrt / PET → UTCI	gap
urban_heat	`lst.pipeline.calculate_lst()` / `lst.smw`	RS_Imagery / LandCover → LST	implemented
urban_heat	`solweig.pipeline.calculate()`	DSM → SVF → Tmrt → UTCI / PET; Meteorology → Tmrt / PET / UTCI	implemented
urban_heat	`calculate_nbs_coverage()`	UTCI / PET → NBS	gap
mainMap	`feature_form()` / `feature_delete()`	(tooling: edits State objects via `save()`)	implemented
mainMap	`journal()` / `restore_backup()`	(tooling: what-if restore point)	implemented
importer	`_prepare_features()` / `_enrich_rows()` (KEA)	(stored inputs; no edge)	implemented

Notes on use

This document is generated from a manual audit of the codebase at the commit referenced in the accompanying `TODO.md`. Line numbers will drift as the code evolves; re-verify against `grep -n "def save"` in the relevant module before citing a location in a manuscript. The status tags mirror the coverage figures reported in the project's DPSIR gap analysis: of 103 edges in `core/DAG.dot` (one,

Total_Water_Prod → OPEX, is listed twice), 31 are backed by real derivation logic as of this audit; the three added since the original audit are LandCover → Infiltration, the new Soil_Type → Infiltration, and New_Units → LandCover for buildings drawn in the map editor.

Beyond the CBS 85173NED forecasts used for the Population_Growth → Total_Population projection, the external data catalog also offers a finer-grained, Google Earth Engine-hosted population source, the High Resolution Settlement Layer (HRSL) [2], for direct raster ingestion where a sub-neighbourhood estimate is preferable to the CBS city-level forecast.

References

[1] Centraal Bureau voor de Statistiek (CBS). *Regionale prognose 2023–2050* (Table 85173NED). https://opendata.cbs.nl/.

[2] Facebook Connectivity Lab and Center for International Earth Science Information Network (CIESIN), Columbia University (2016). *High Resolution Settlement Layer (HRSL).* Source imagery for HRSL Copyright 2016 DigitalGlobe. https://gee-community-catalog.org/projects/hrsl/.

[3] Lambert, A., Brown, T. G., Takizawa, M., & Weimer, D. (1999). A review of performance indicators for real losses from water supply systems. *Journal of Water Supply: Research and Technology-AQUA*, 48(6), 227–237. (International Water Association Water Loss Task Force ILI methodology.)

[4] Demographia. *International Housing Affordability Survey* — Median Multiple methodology (median house price / median household income). http://www.demographia.com/.

[5] Lindberg, F., Holmer, B., & Thorsson, S. (2008). SOLWEIG 1.0 – Modelling spatial variations of 3D radiant fluxes and mean radiant temperature in complex urban settings. *International Journal of Biometeorology*, 52(7), 697–713.

[6] Ermida, S. L., Soares, P., Mantas, V., Göttsche, F.-M., & Trigo, I. F. (2020). Google Earth Engine open-source code for Land Surface Temperature estimation from the Landsat series. *Remote Sensing*, 12(9), 1471. https://doi.org/10.3390/rs12091471.

[7] Oke, T. R. (1987). *Boundary Layer Climates* (2nd ed.). Routledge. (Urban canyon height-to-width aspect ratio.)

[8] Matzarakis, A., & Mayer, H. (1996). Another kind of environmental stress: thermal stress. *WHO Newsletter*, 18, 7–10. (Physiological Equivalent Temperature classification.)

[9] Koopmans, S., Heusinkveld, B. G., & Steeneveld, G. J. (2020). A standardized Physical Equivalent Temperature urban heat map at 1-m spatial resolution to facilitate climate stress tests in the Netherlands. *Building and Environment*, 181, 106984.

[10] Höppe, P. (1999). The physiological equivalent temperature – a universal index for the biometeorological assessment of the thermal environment. *International Journal of Biometeorology*, 43(2), 71–75.

[11] Walther, E., & Goestchel, Q. (2018). The P.E.T. comfort index: Questioning the model. *Building and Environment*, 137, 1–10.

[12] Hersbach, H., Bell, B., Berrisford, P., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049.

[13] Alduchov, O. A., & Eskridge, R. E. (1996). Improved Magnus form approximation of saturation vapor pressure. *Journal of Applied Meteorology*, 35(4), 601–609.

[14] SOLWEIG-GPU, version 2.0.0. GPU-accelerated SOLWEIG thermal comfort model (Python, PyTorch). `https://github.com/nvnsudharsan/solweig-gpu`, `https://solweig-gpu.readthedocs.io`.

[15] Bröde, P., Fiala, D., Błażejczyk, K., et al. (2012). Deriving the operational procedure for the Universal Thermal Climate Index (UTCI). *International Journal of Biometeorology*, 56(3), 481–494.

[16] USDA Natural Resources Conservation Service (1986). *Urban Hydrology for Small Watersheds*. Technical Release 55 (TR-55), 2nd ed. (Curve number tables for antecedent moisture condition II.)

[17] USDA Natural Resources Conservation Service (2009). *National Engineering Handbook, Part 630 Hydrology, Chapter 7: Hydrologic Soil Groups*. (Group definitions, final infiltration rates, dual groups for a water table within 60 cm.)

[18] USDA Natural Resources Conservation Service (2004). *National Engineering Handbook, Part 630 Hydrology, Chapter 10: Estimation of Direct Runoff from Storm Rainfall*. (Runoff equation, $I_a = 0.2S$.)

[19] Wageningen Environmental Research. *Bodemkundig Informatie Systeem (BIS Nederland)*: Bodemkaart 1:50 000 (2025) and groundwater-level grids (GHG, GLG, GVG). `https://bodemdata.nl/`.

[20] Bonafoni, S., Sekertekin, A. (2020). Albedo retrieval from Sentinel-2 by new narrow-to-broadband conversion coefficients. *IEEE Geoscience and Remote Sensing Letters*, 17(9), 1618–1622.

[21] Oke, T.R. (1987). *Boundary Layer Climates*, 2nd ed. Routledge. (Table 1.1 albedo ranges, Table 2.1 thermal properties.)

[22] Oke, T.R., Mills, G., Christen, A., Voogt, J.A. (2017). *Urban Climates*. Cambridge University Press.

[23] European Environment Agency. *CORINE Land Cover nomenclature*. `https://land.copernicus.eu/`.

[24] European Commission. *INSPIRE Data Specification on Land Use (D2.8.III.4)*, Hierarchical INSPIRE Land Use Classification System (HILUCS). `http://inspire.ec.europa.eu/codelist/HILUCSValue`.